AI SYSTEMS ENGINEERING · WEEK 5

Ranking Systems

Hundreds of millions of candidates, twenty on the page.
The architecture, the data, and the measurement.

Tonight

1. **Failure of the Week** — Facebook's feed learned what it was measured on
2. **Ranking systems** — a video feed: its architecture, its data, its measurement
3. **Design Studio** — design the simplified Amazon store front
4. **Lab** — publish the API: a chat route and the triage step behind a public URL, and call each other's

FAILURE OF THE WEEK

Meaningful social interactions

- 2018: Facebook re-ranks the News Feed for **meaningful social interactions**: posts people talk about.
- The ranker's target is a **weighted sum of what people do** with a post. A comment or a reshare counts for many times a like.
- Engagement rises. In 2019 Facebook's own researchers report the favoured posts are **angrier**.
- Publishers and political parties tell Facebook they have **changed what they write** to fit the ranking.

30 : 1

a reshare against a like, in the ranking formula, as reported

Source: Hagey and Horwitz, The Wall Street Journal, 15 September 2021, from Facebook's internal documents.

FAILURE OF THE WEEK

Where did the label come from?

From what could be counted within seconds, at scale.

Facebook's stated goal was a conversation worth having. The label was a reaction, a comment, a reshare: events the page writes the moment they happen, billions a day, free.

The ranker learned the label it was given. The label was a proxy, and the proxy had a price: the content that earns comments fastest is the content that makes people angry.

Three facts about the data:

- The **label** was engagement, chosen because it arrives in seconds. What Facebook said it wanted arrives in surveys, slowly, in thousands of rows, not billions.
- The **rows** were what the feed had shown. A post the ranker never surfaced earned nothing, and taught nothing.
- The **producers adapted.** The training set after the change was written by people who had learned the new weights.

HOLD THIS THOUGHT

What it cost

- internal research in 2019 and 2020 documented the effect; proposed changes to the weights were argued over against the engagement they would cost
- the documents became public in 2021, in the Wall Street Journal's Facebook Files, and are the example every regulator now cites
- Facebook says the weights have been revised many times since

What a fix would have needed:

- a **label that is the goal**, measured on a sample, even slowly: a survey answer joined to what the viewer was shown
- a **guardrail in the release gate** that can stop a change whose primary metric rose
- a **measurement of the loop**: how the candidates change after the ranking changes

Each is a component. None is a model.

Where we are

#	STEP	SO FAR	TONIGHT
1	Scope & requirements	requirements as rates the three numbers	a feed's requirements
2	Frame the AI task	the model's contract: inputs to outputs	an ordering of twenty, per viewer, per request
3	Metrics	rates with intervals · the noise floor comparing versions · judges precision at *k* · the holdout	recall at *k* · NDCG the A/B test · interleaving guardrails
4	Data	the golden set the tables · the point-in-time join	the impression log · implicit labels position bias · the index as data
5	Architecture	placement · the queue · the cascade the batch and online scorer	the index · two towers · the filters what is computed before the request
6	Deep dive		
7	Failure, scale & operations	the failure path · a failed run	a stale index · a new video · the feedback loop

A VIDEO FEED

Ranking Systems

An ordering of a few videos out of hundreds of millions, for this viewer, right now.

A video feed

A short-video app with 20,000,000 viewers a day and 300,000,000 videos, 2,000,000 of them uploaded today.

One request: a viewer opens the app, or scrolls to the end of a page. The system returns **twenty videos, in order**.

Two things make this hard:

- the right twenty depend on **who is asking** and **what they just watched**
- the viewer is **waiting**

THE NUMBERS	
videos	300,000,000
viewers	20,000,000 a day
requests at the peak	3,000 a second
the page	20 videos, within 300 ms
ground truth	watched, skipped, liked, shared; known within seconds

The interview versions of this system: "design the home feed"; "design product search for an online store"; "design people you may know."

Elements of designing this system

For this system, as for every system:

- **Architecture.** Where the models run, what surrounds them, what runs when they fail.
- **Data handling.** Where the dataset comes from, how it is kept, what it specifies.
- **Measurement.** How you know the order is any good.

The models are **treated as given**: inputs to outputs, with a datasheet. Everything else is what we build.

The architecture

What shapes the design

- There are **300,000,000 candidates** and **20 slots**. The work is choosing, then ordering what was chosen.
- The strongest signal is **who is asking and what they just watched**. It exists only at the request.
- The page has **300 ms**, and there are 3,000 requests a second at the peak.
- **Position is part of the output.** Slot one is seen by everyone who opened the page; slot twenty by few.
- The truth arrives **seconds later**, for the twenty that were shown, and for no others.

The obvious design: a nightly list for every viewer



The nightly list: last week's scoring system, applied to recommendations. Stable, cheap, and a day old.

THE LIST	
what it gets right	stable; cheap to run; one job, one table, one read; the page is a lookup
what it cannot do	know what the viewer watched a minute ago; show a video uploaded this morning; serve a viewer it has never scored

What does the request know that last night could not?

The problem: a number per pair

a scoring system: one number per account

account 1	0.81
account 2	0.07
account 3	0.62
account 4	0.15
account 5	0.44
account 6	0.09

400,000 numbers · computed at night

a ranking system: one number per viewer and video

	video 1	video 2	video 3	video 4	video 5	video 6
viewer 1	0.9	0.1	0.4	0.2	0.7	0.1
viewer 2	0.2	0.8	0.1	0.6	0.1	0.3
viewer 3	0.1	0.2	0.9	0.1	0.3	0.6
viewer 4	0.5	0.1	0.2	0.8	0.2	0.1
viewer 5	0.3	0.6	0.1	0.2	0.9	0.2
viewer 6	0.1	0.3	0.7	0.1	0.2	0.8

20,000,000 viewers × 300,000,000 videos = 6×10^{15} pairs, one inference each

every new video is 20,000,000 new pairs: every viewer, recomputed

2,000,000 new videos a day · and the viewer half of every pair changes with every watch

A score belongs to an account. A ranking belongs to a pair; the pairs cannot be computed ahead of time, and nothing about them stays computed.

The problem: a number per pair

a scoring system: one number per account

account 1	0.81
account 2	0.07
account 3	0.62
account 4	0.15
account 5	0.44
account 6	0.09

400,000 numbers · computed at night

a ranking system: one number per viewer and video

	video 1	video 2	video 3	video 4	video 5	video 6
viewer 1	0.9	0.1	0.4	0.2	0.7	0.1
viewer 2	0.2	0.8	0.1	0.6	0.1	0.3
viewer 3	0.1	0.2	0.9	0.1	0.3	0.6
viewer 4	0.5	0.1	0.2	0.8	0.2	0.1
viewer 5	0.3	0.6	0.1	0.2	0.9	0.2
viewer 6	0.1	0.3	0.7	0.1	0.2	0.8

20,000,000 viewers × 300,000,000 videos = 6×10^{15} pairs, one inference each

every new video is 20,000,000 new pairs: every viewer, recomputed

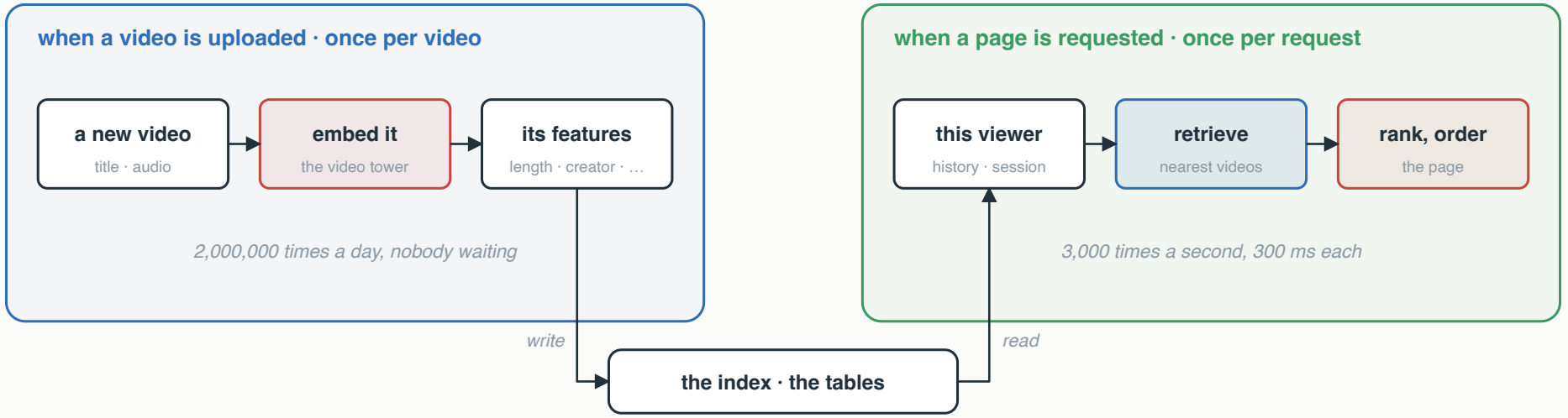
2,000,000 new videos a day · and the viewer half of every pair changes with every watch

A score belongs to an account. A ranking belongs to a pair; the pairs cannot be computed ahead of time, and nothing about them stays computed.

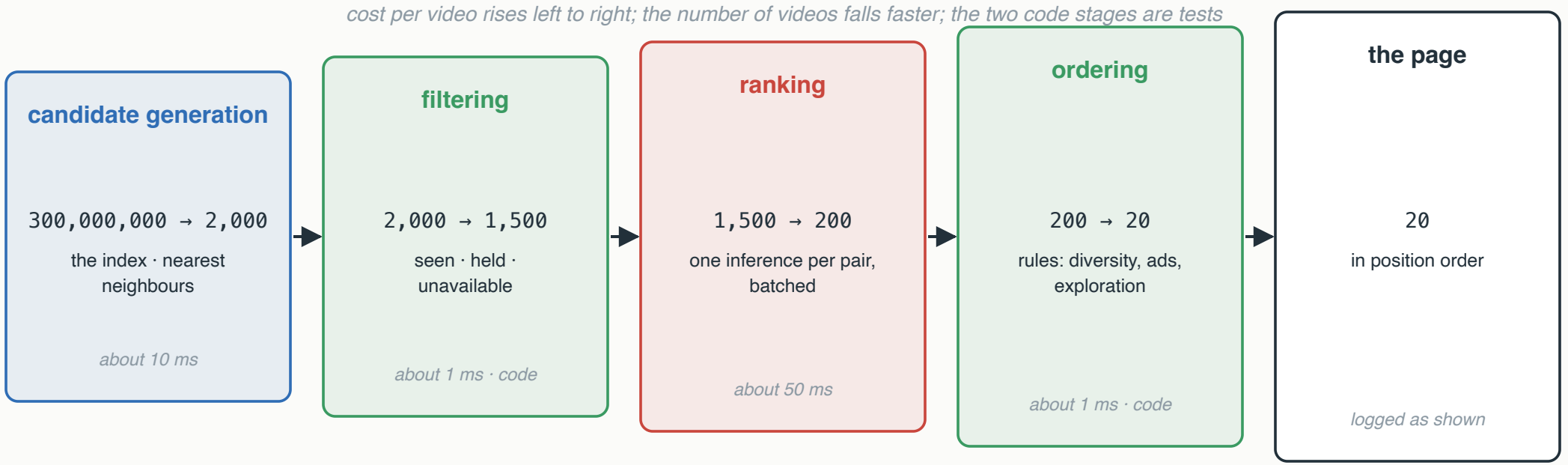
How should we handle so many to compute?

Resolution: compute each half when it becomes known

Compute the video's part when the video is uploaded. Compute the viewer's part as part of the page request.



The cascade for a feed

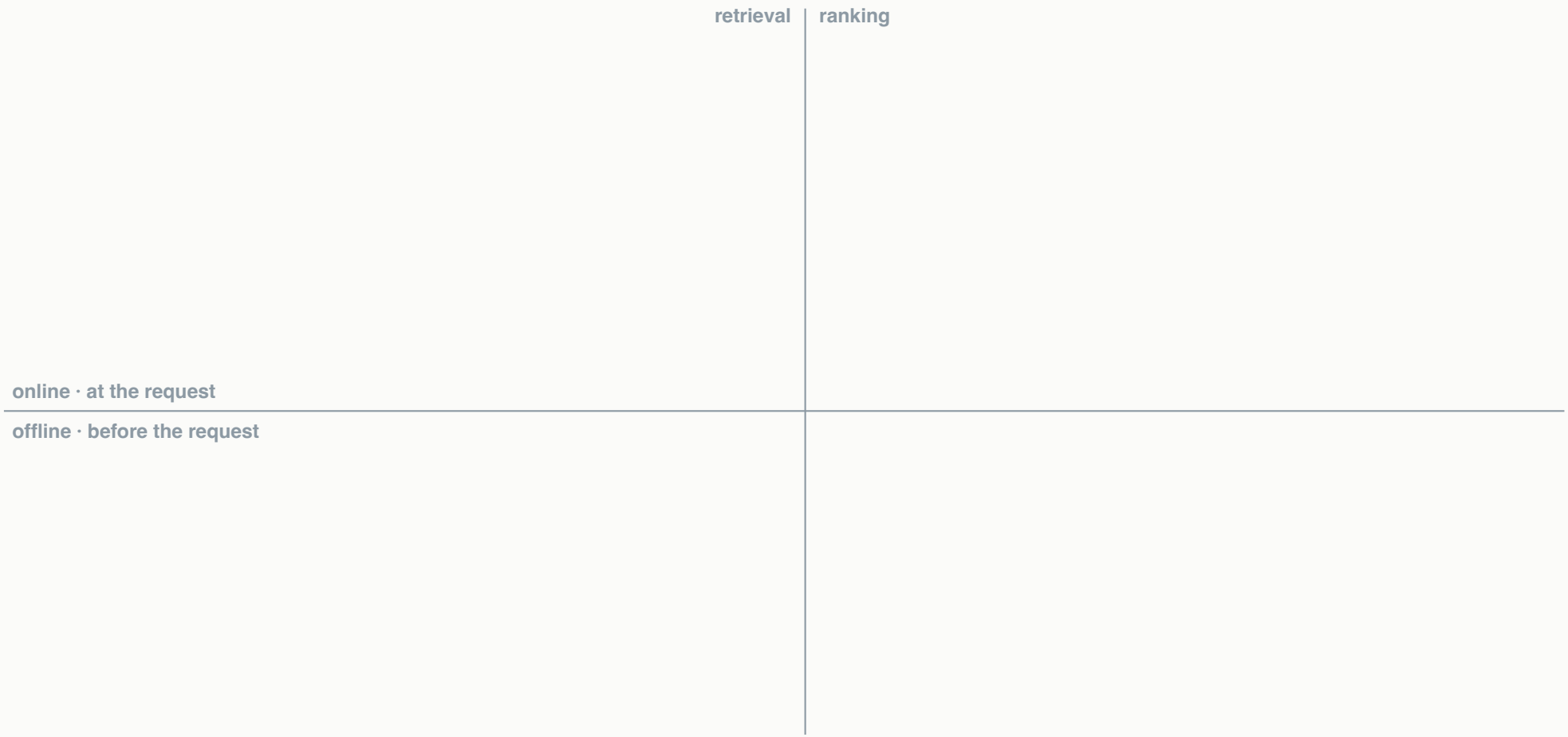


The ranker sees only what candidate generation returned and the filters let through. Counts and times illustrative.

$$\text{cost per request} = N \cdot c_{\text{candidate generation}} + k \cdot c_{\text{ranking}}, \qquad k \ll N$$

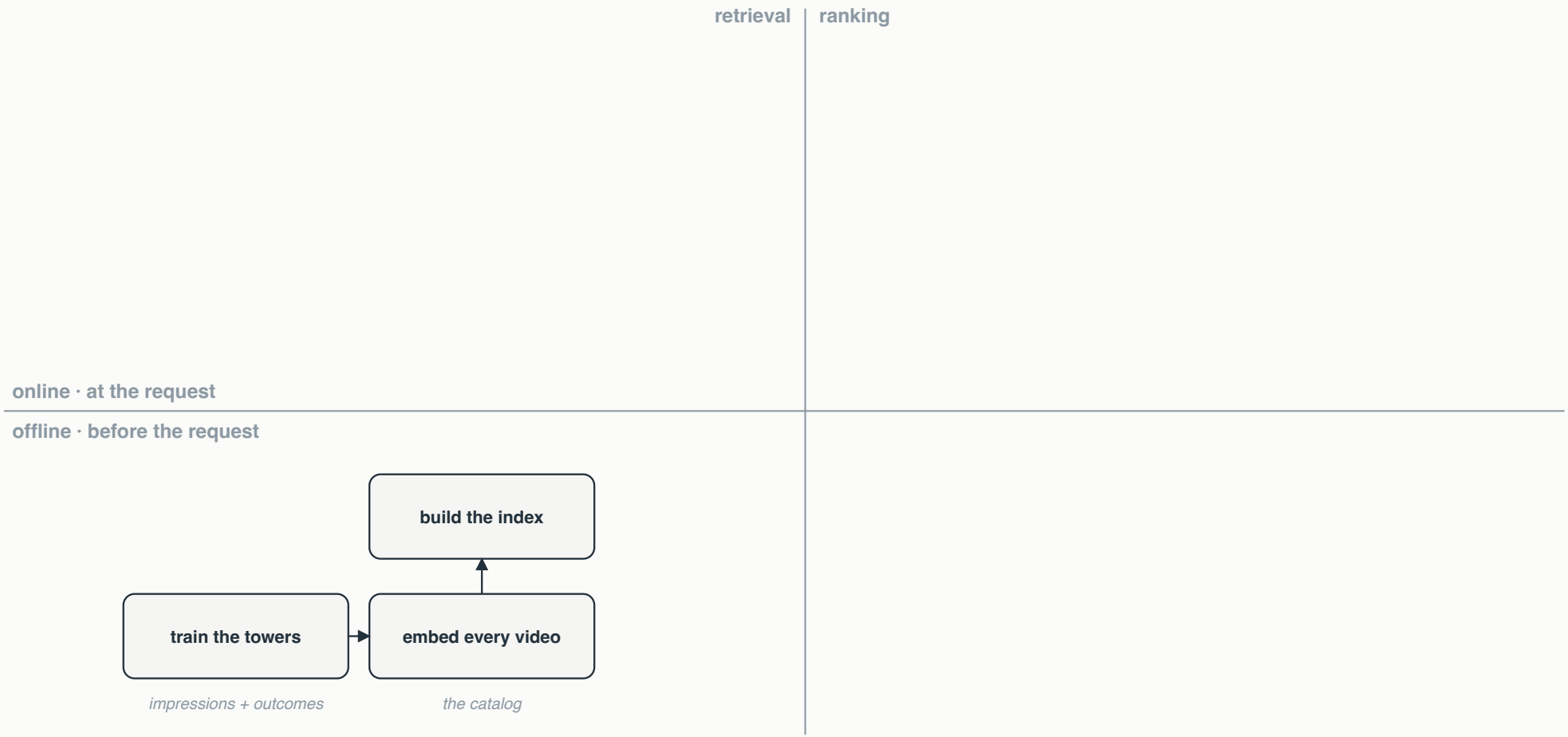
Blue: candidate generation, all $N = 300{,}000{,}000$ videos through the index, a tiny cost each · red: ranking, one inference for each of the $k \approx 1{,}500$ candidates that passed the filters · the two green stages are code and are left out.

The ranking system



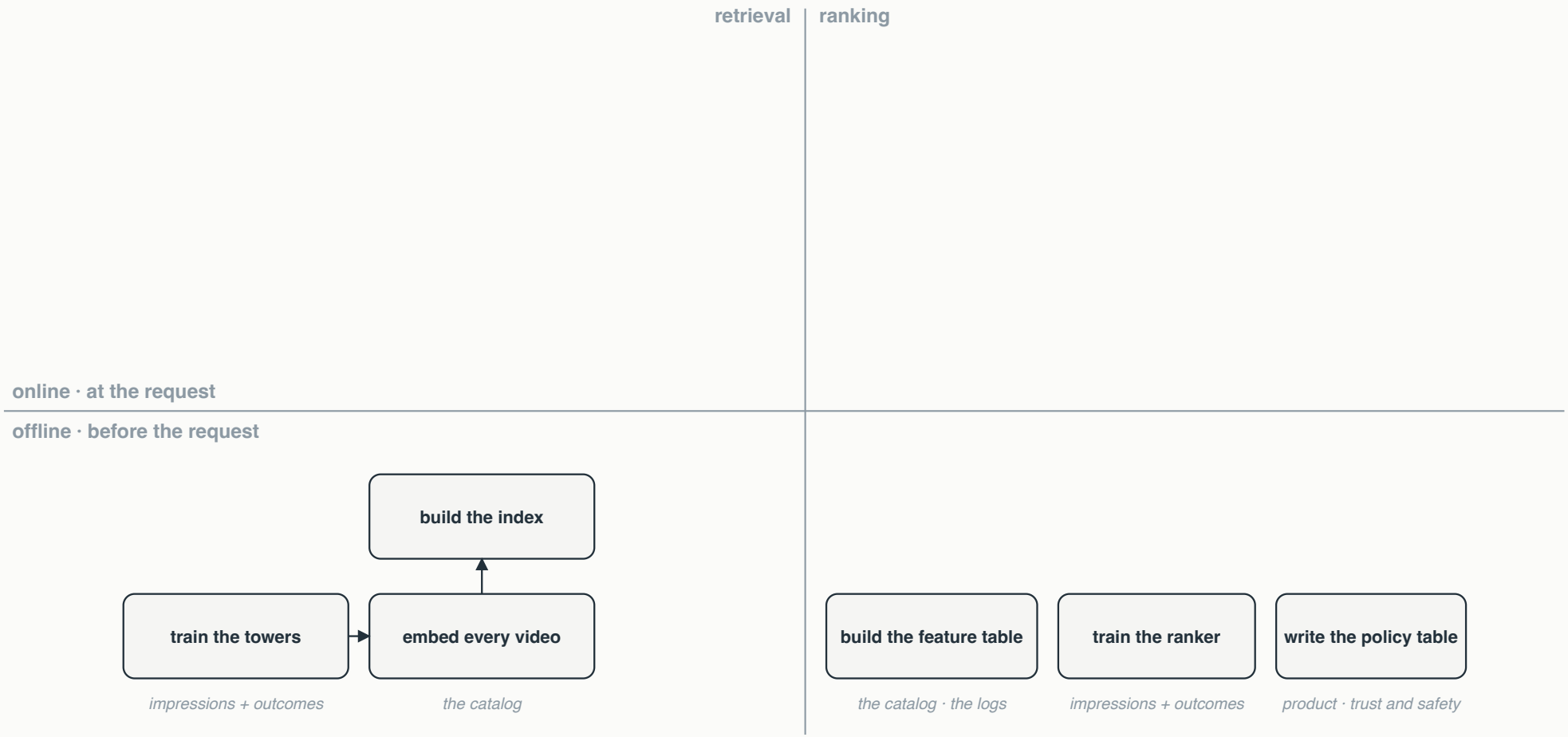
Above the line: what runs while the viewer waits. Below: what ran before. Left: finding candidates. Right: ordering them.

The ranking system: offline, the retrieval side



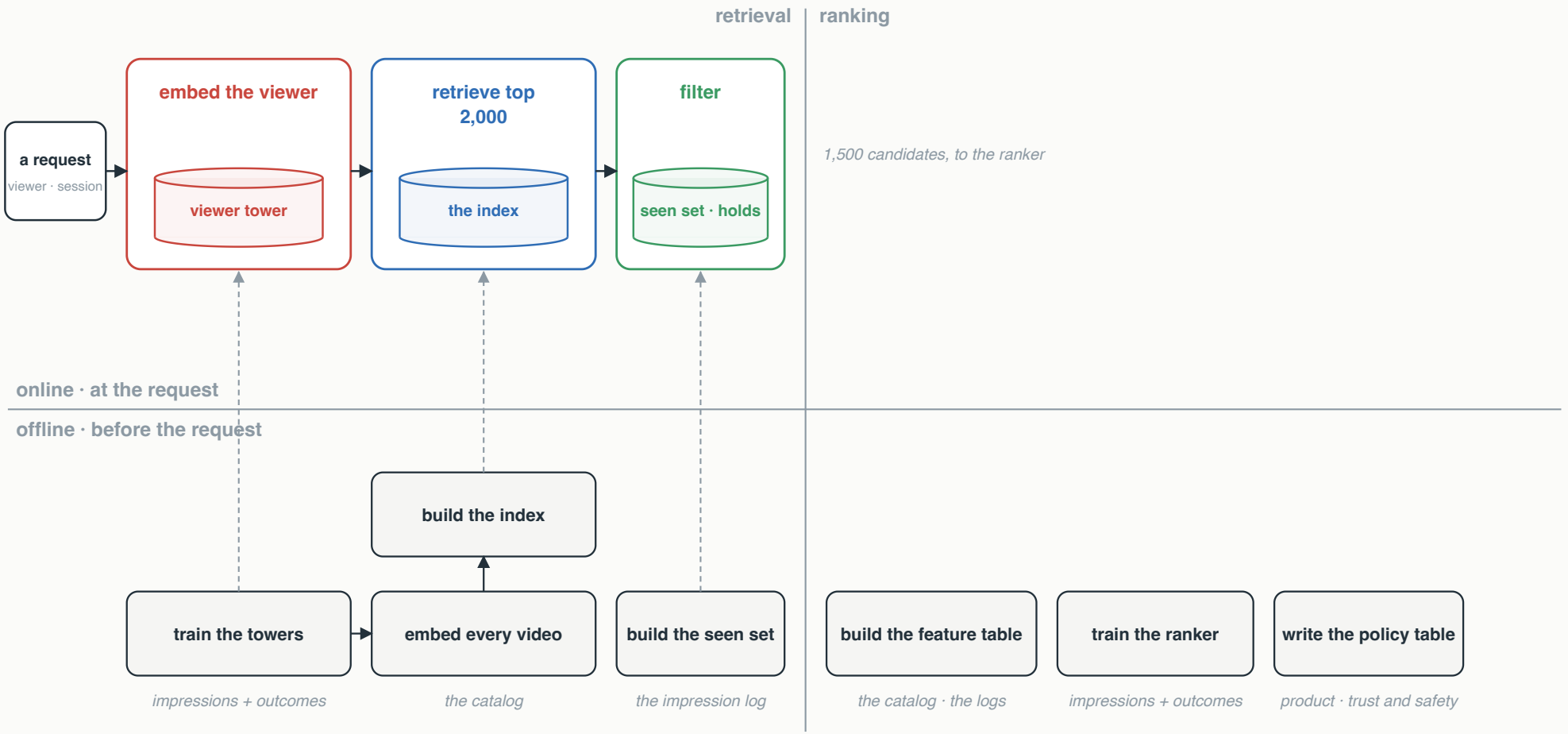
Offline, the retrieval side: the towers are trained, every video is embedded, the index is built.

The ranking system: offline, the ranking side



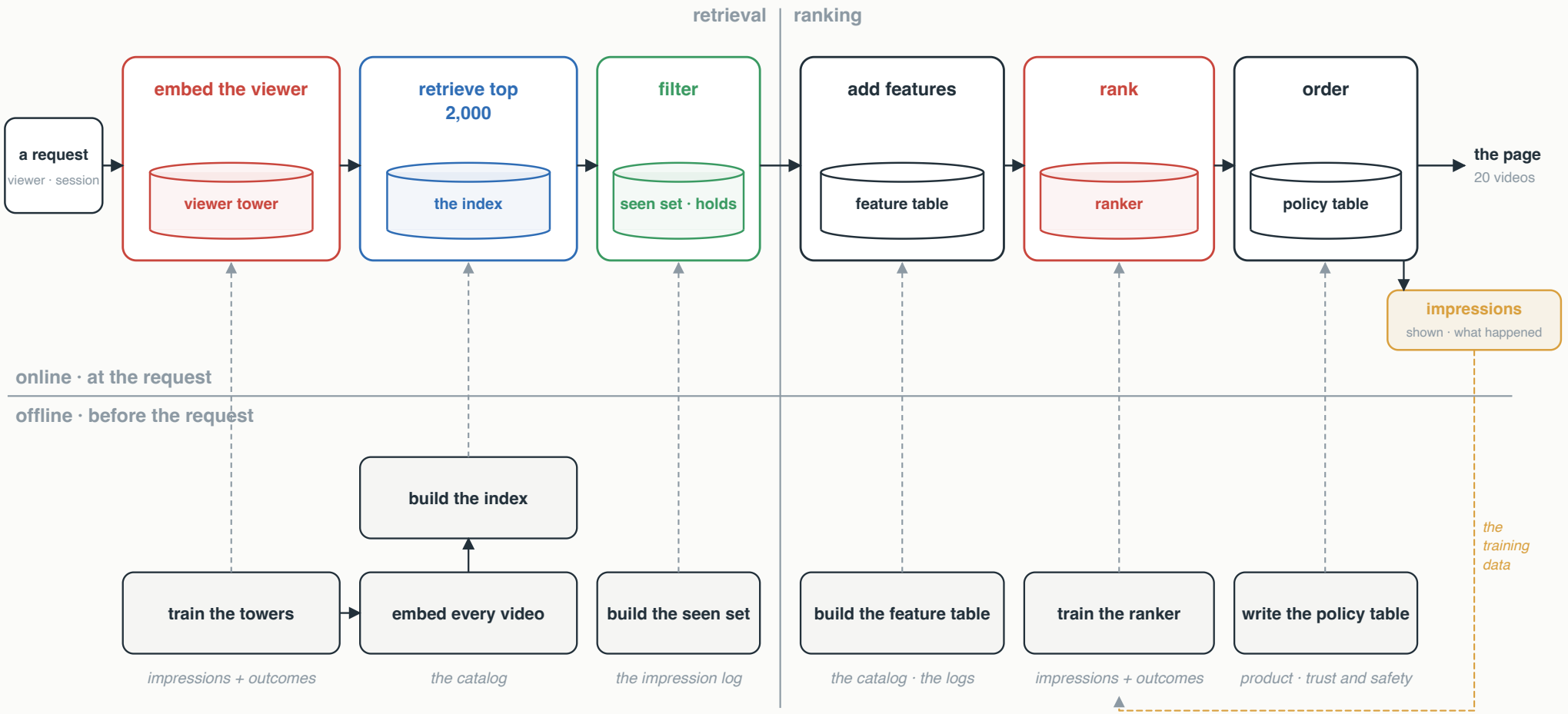
Offline, the ranking side: the feature table is built, the ranker is trained, the policy table is written by people.

The ranking system: online, retrieval and the filters



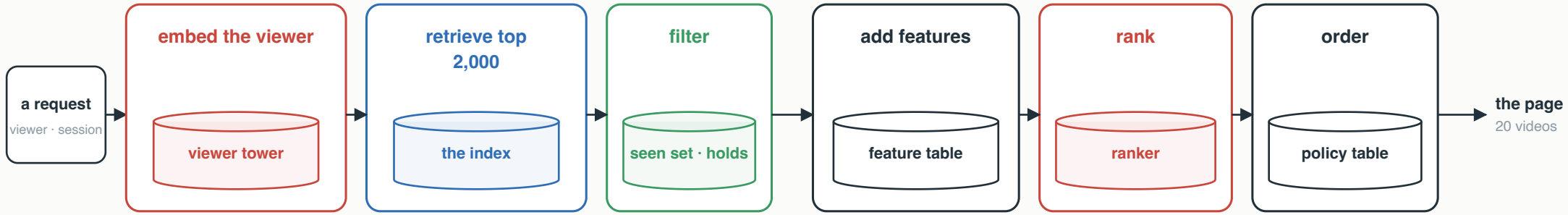
Online, the retrieval side: the viewer is embedded, the nearest 2,000 are retrieved, the cheap filters run before any inference.

The ranking system: online, ranking and ordering



The whole system. Every online stage reads an artifact that the offline side built; the impression log is what the offline side trains on.

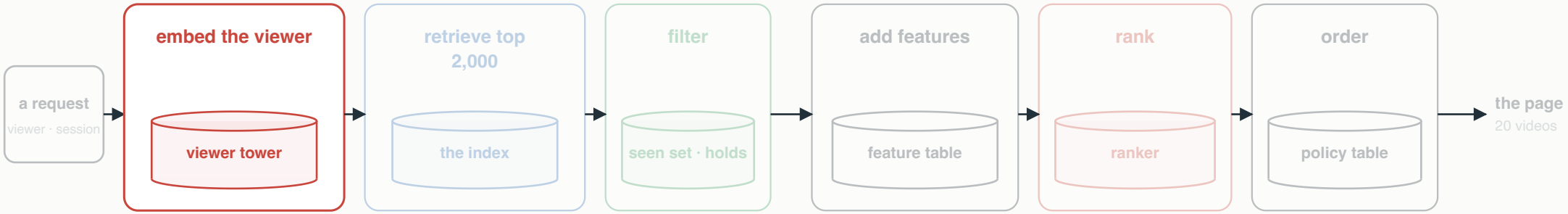
The inference path



the inference path: everything the viewer waits for, left to right, inside 300 ms

The top row of the diagram, on its own. Each stage in turn, left to right, with the artifact it reads and where that artifact came from.

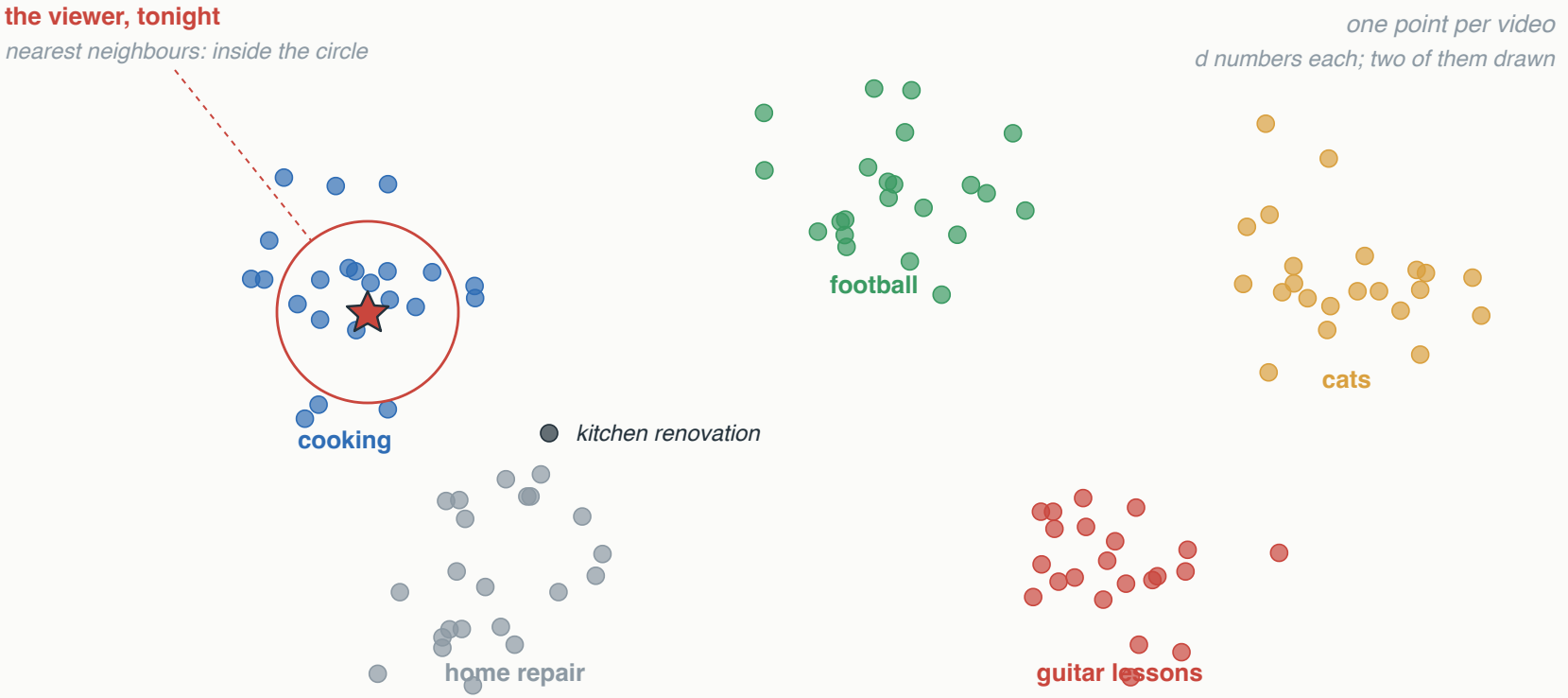
The inference path: embed the viewer



the inference path: everything the viewer waits for, left to right, inside 300 ms

How is one viewer turned into something an index can search with?

Embeddings: a learned map



An embedding is a learned map: videos watched by the same people land near each other. A viewer is a point in the same space.

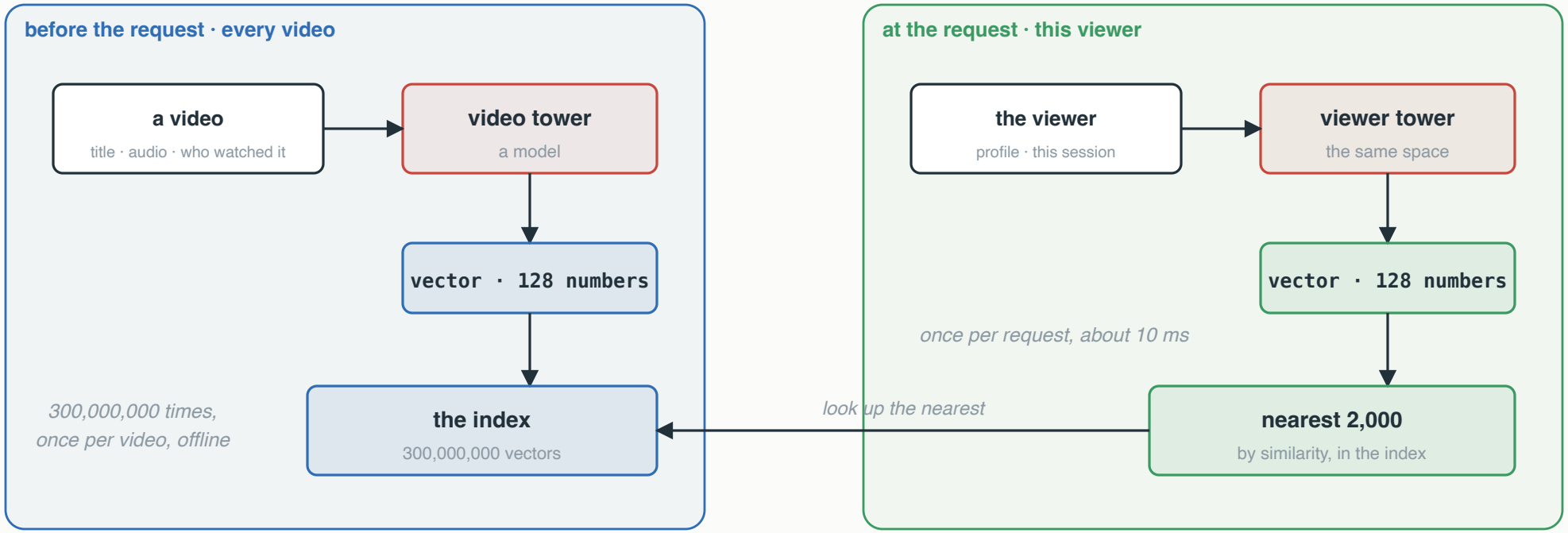
The embedding model as a component

THE MODEL	
input features	a video: title, audio, who watched it · a viewer: their watch history
output	an **embedding**: a dense vector of *d* floats, a learned representation of the input
the embedding space	one space, shared by videos and viewers; **cosine similarity** (or the inner product) between two embeddings is the learned relevance; a single embedding carries no meaning on its own

PROPERTIES	
model version	embeddings from two versions are not comparable; a new version means **re-embedding the whole catalog** and rebuilding the index
inference cost	one forward pass per video, once, offline; one per request for the viewer
limits	the dimensions are not interpretable; an item with no interactions has no useful embedding (the **cold-start** problem)

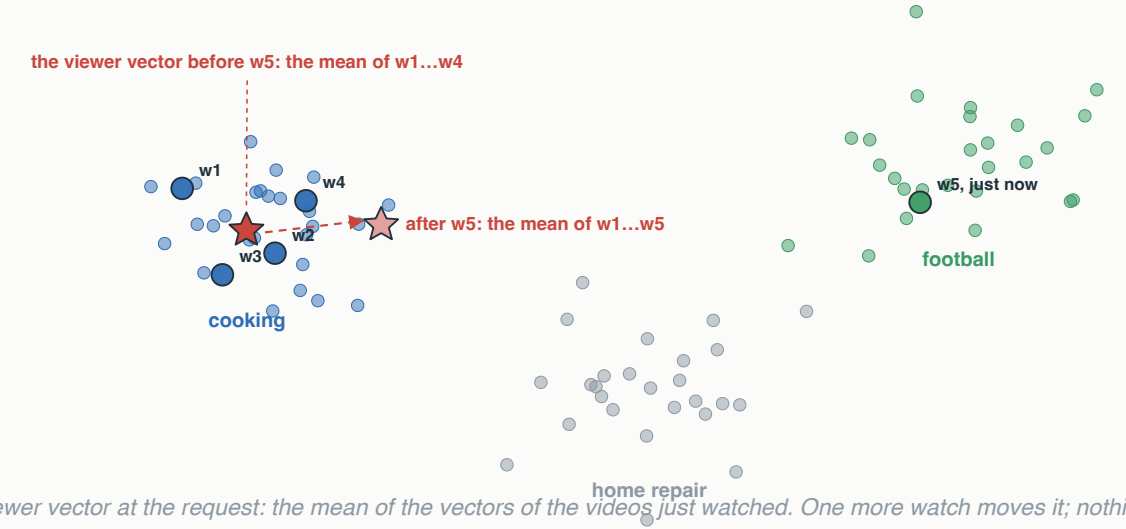
d: the embedding dimensionality, 64 to 1,024 in practice; chosen with the model.

Two towers



Two towers: the pair score becomes a similarity between two vectors, so the video half can be computed once, ahead of time.

The viewer vector, at the request

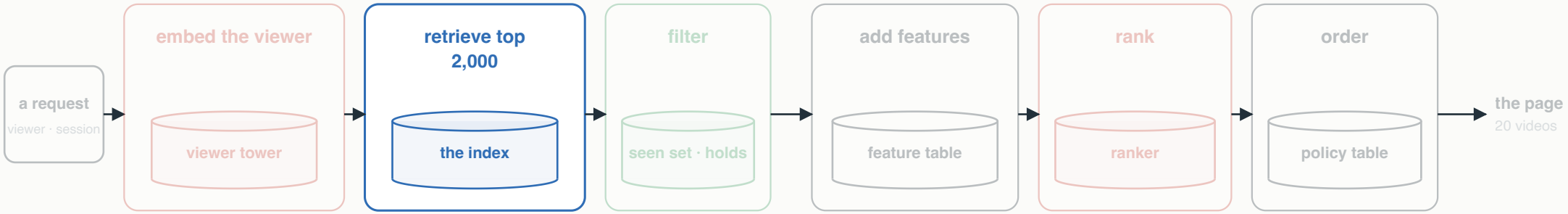


The viewer vector at the request: the mean of the vectors of the videos just watched. One more watch moves it; nothing is retrained.

$$u = \frac{1}{n} \sum_{i=1}^{n} v_i$$

u: the viewer vector · v_i: the vector of the i-th most recent watch, read from the index · n: the watches that count, 20 to 50 in practice.

The inference path: retrieve



the inference path: everything the viewer waits for, left to right, inside 300 ms

How do you find 2,000 videos out of 300,000,000 in ten milliseconds?

The index

What it is. A structure built offline over every video's vector that answers one question: *which stored vectors are nearest to this one?* Built once an hour; read 3,000 times a second.

What it is for. The same job an index in a database does for a key, done for *nearness* instead of equality.

The scan it replaces:

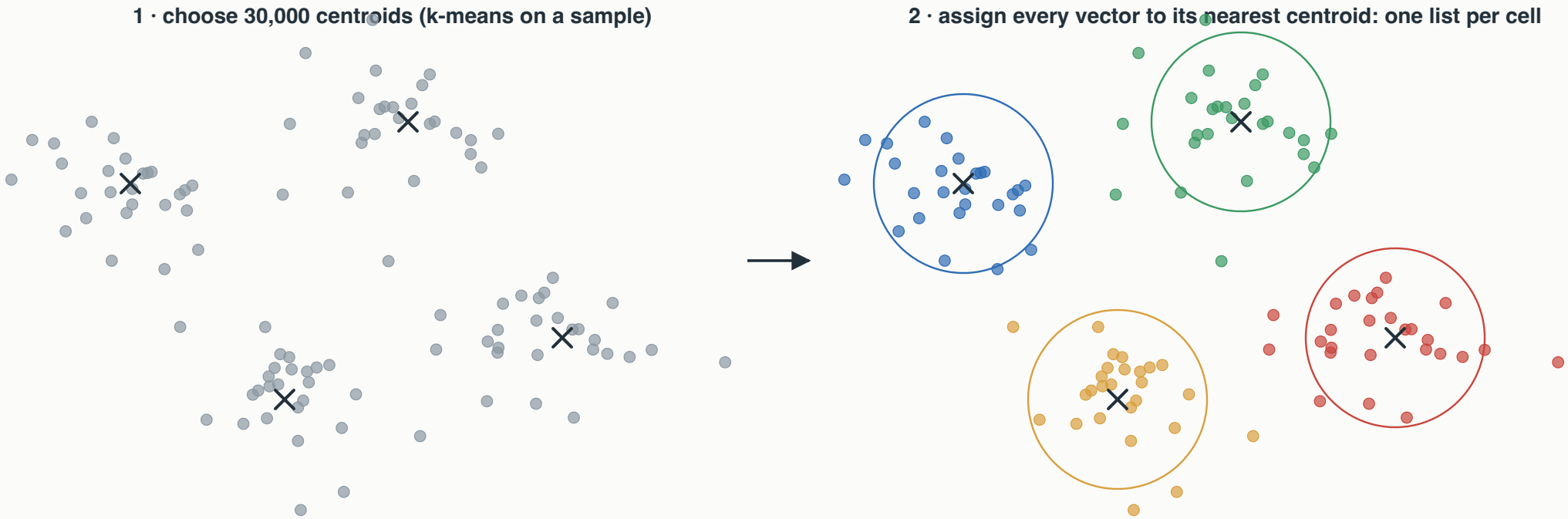
$$300{,}000{,}000 \times 128 \approx 4 \times 10^{10} \text{ multiplications per request}$$

about a second on one core, for one viewer. The page has 10 ms for this stage.

So the index does not look at every vector. It looks at a few, chosen well, and returns *approximately* the nearest. How, next.

128: the vector length d · similarity: the inner product of the two vectors, one multiplication per number.

Building the inverted file

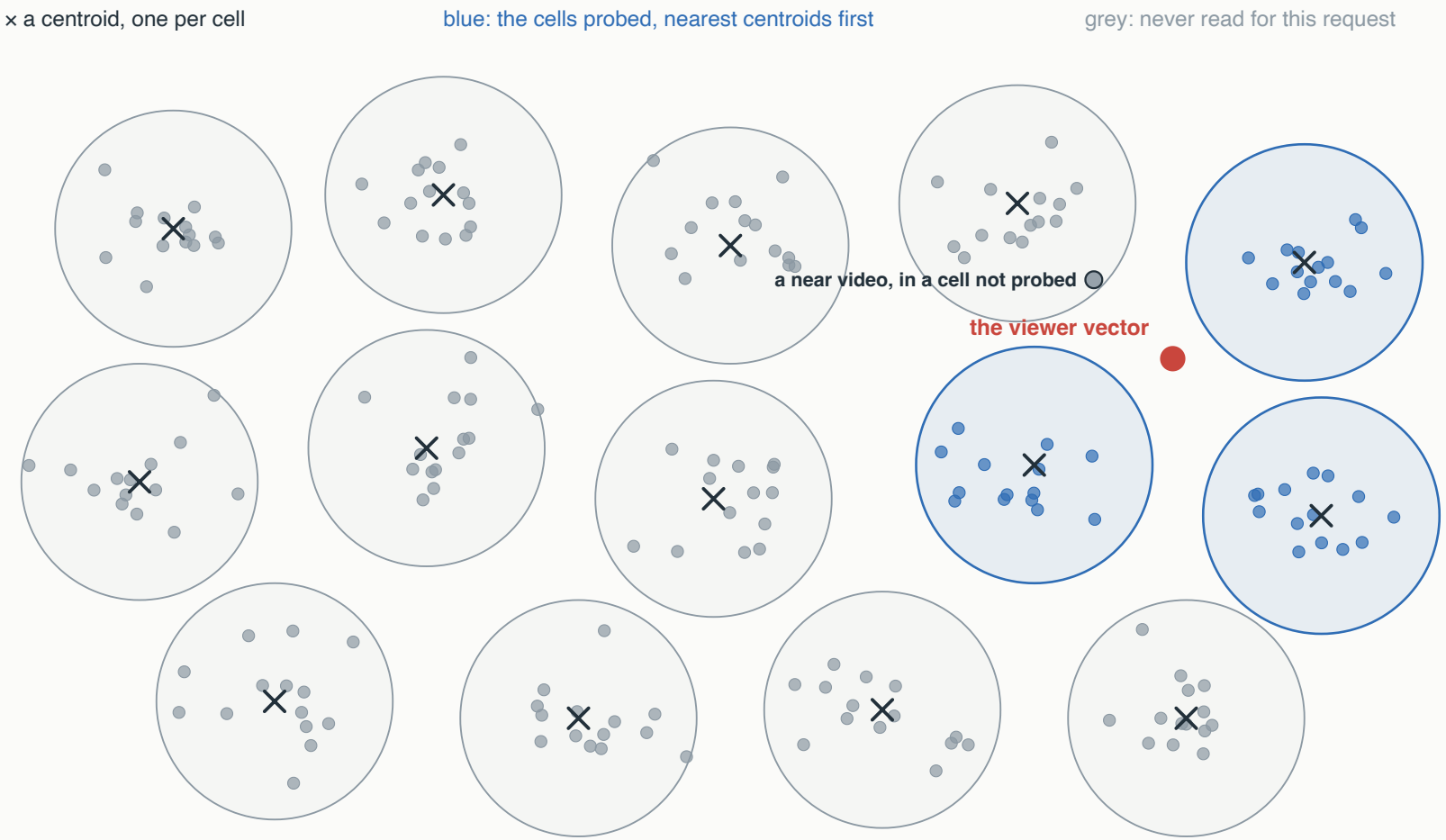


the index = the centroid table (30,000 × 128 numbers, tiny) + one list of video ids and vectors per cell (300,000,000 in all)

built offline, once an hour, by the job with the check and the pointer

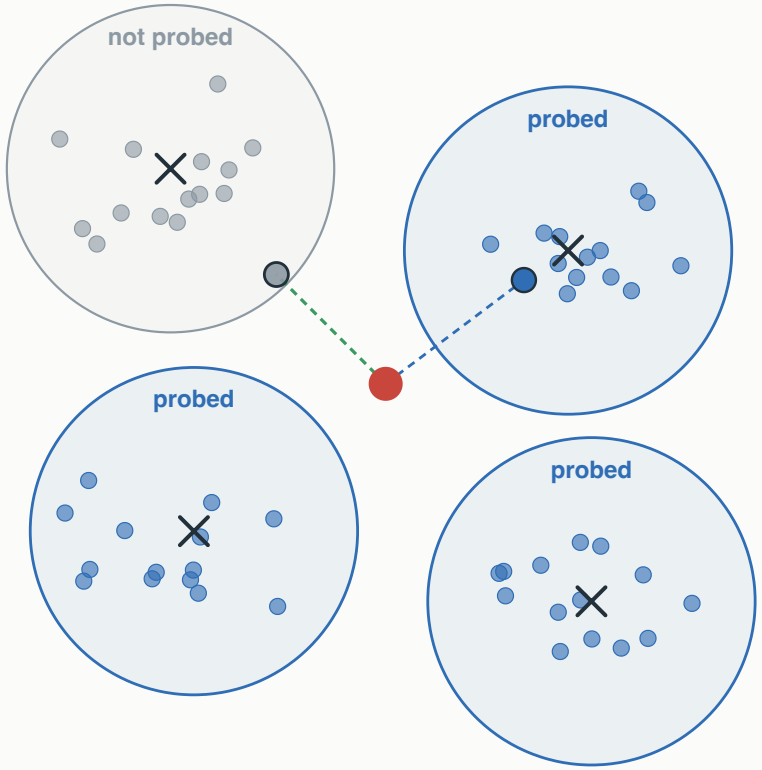
Building the inverted file. Four cells drawn; the real index has thirty thousand.

Searching it: probe the nearest cells



At the request: compare with the 30,000 centroids, open the nearest cells, compute the similarity to every vector in them, keep the top 2,000.

The miss: a neighbour across the boundary



the viewer vector

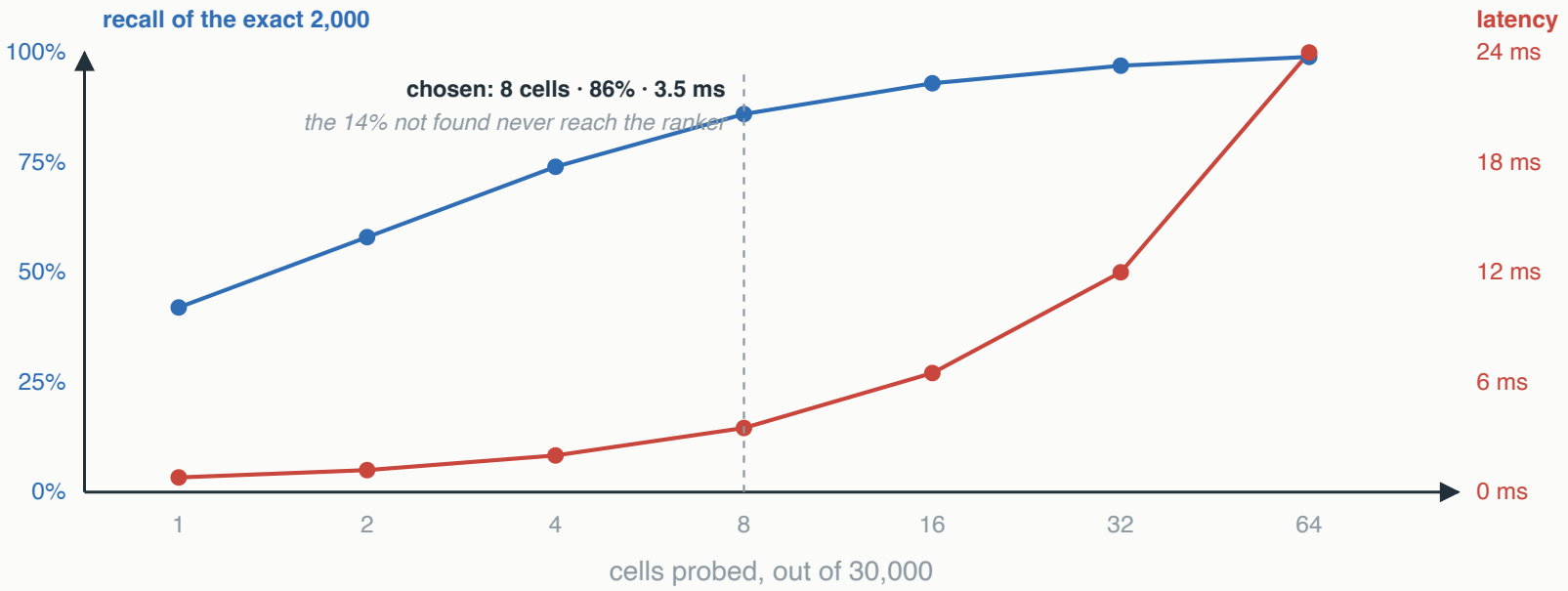
the best the scan found
similarity rank 1 among the three cells

the grey video
nearer to the viewer vector,
never scanned: its cell was not opened

its centroid is the 4th nearest.
A 4th probe would find it, at the cost of scanning 10,000 more vectors.

The nearest video sits across a cell edge, in a cell that was never opened. Nothing fails; the answer is a little worse.

The dial: cells probed



The index trades recall for latency, and the number it gives up is measured offline against an exact scan. Values illustrative.

A vector database

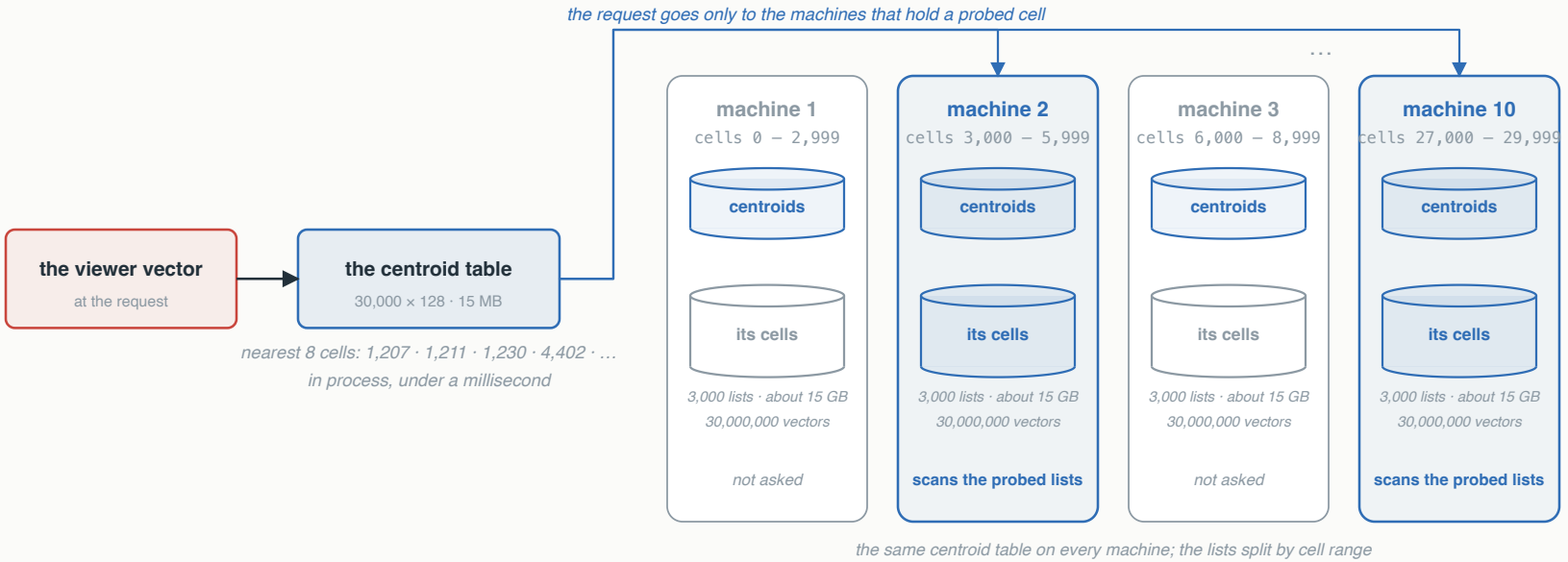
Holding these vectors and this index, and keeping them searchable while they change, is the core idea of a vector database. The index is the mechanism; the database is everything a product wraps around it:

WHAT IT ADDS	WHY THE INDEX ALONE CANNOT
insert, update, delete without a full rebuild	a new video lands in a cell at once; a deleted one leaves its list
filtering by metadata, inside the search	"nearest, among videos in this language, not under a hold" without scanning the rest
persistence and replication	the lists survive a restart and are served from more than one machine
a version for the whole thing	the vectors, the centroids and the tower that made them, pinned together

As of 2026: pgvector, Milvus, Qdrant, Weaviate and Pinecone are vector databases; FAISS and hnswlib are index libraries without the rest.

Where the index lives

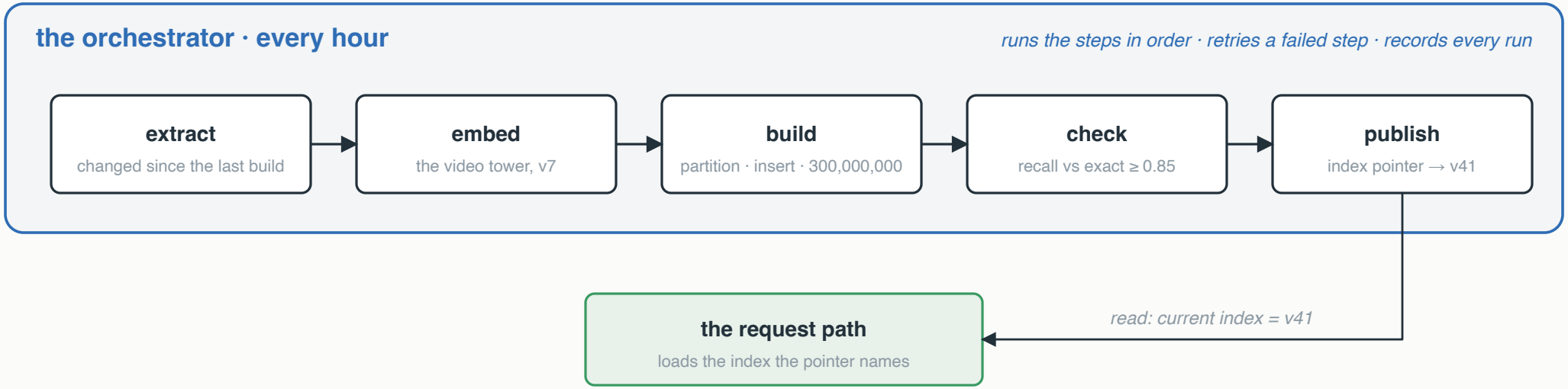
300,000,000 vectors × 128 numbers × 4 bytes ≈ **150 GB**: the lists do not fit one process. The centroid table does: 15 MB.



Sharded by cell: the centroid table is the routing table, replicated everywhere; the lists are spread over machines by cell range.

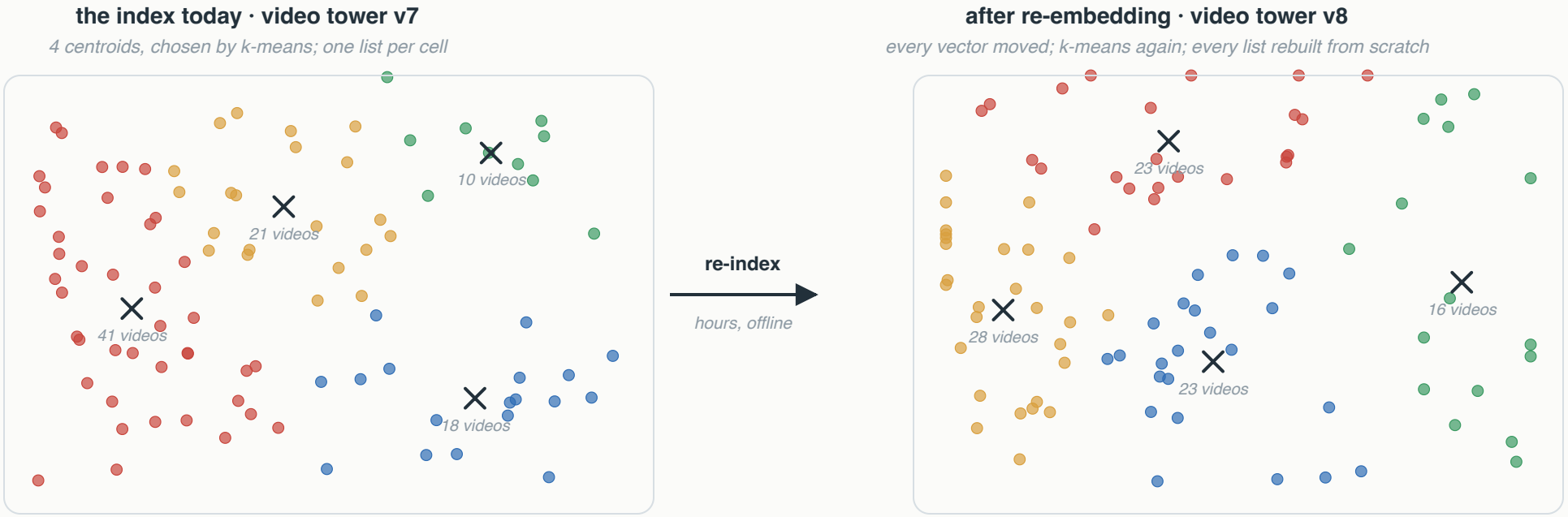
Each machine loads its cell range from the object store at start; the cold start is the index load. One-byte codes per number cut the lists four times, at a measured cost in recall: a second knob on the dial.

Building the index



The index is built like any batch output: a run, a check, a pointer. A build that fails its check leaves v40 in service.

Re-indexing: the centroids are chosen again



nothing carries over: the vectors, the centroids and the lists are all v8; the pointer swaps v7 → v8 in one move, and the viewer tower switches with it

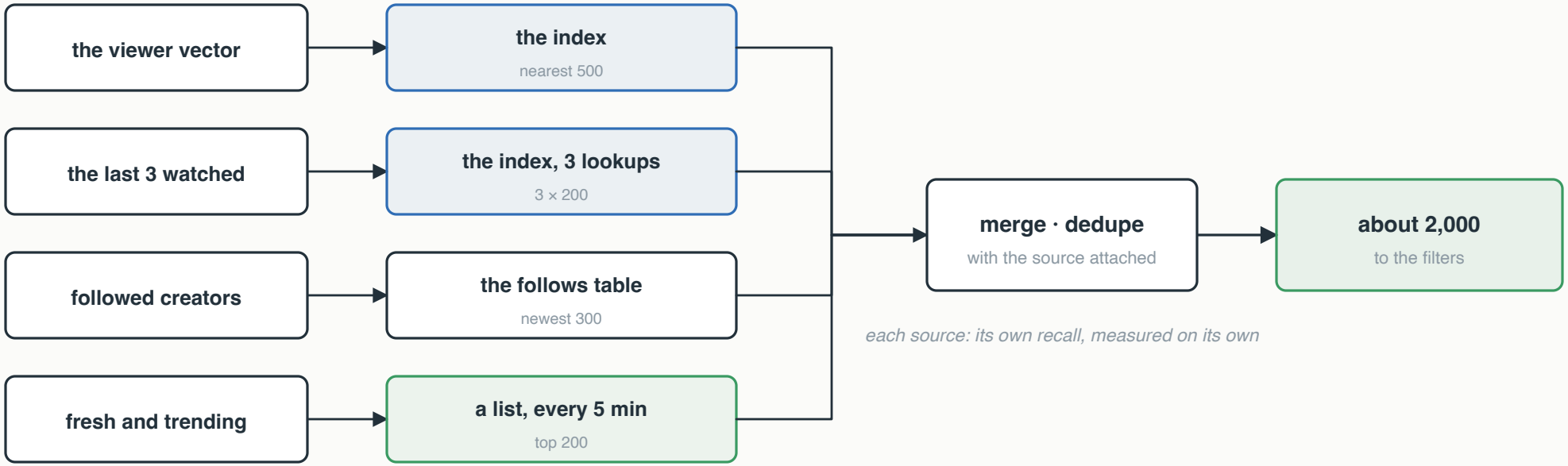
A re-index chooses the centroids again and repopulates every cell. The same happens, more cheaply, when enough new videos have drifted the cells.

As of 2026: the approximate nearest-neighbour approaches

APPROACH	THE IDEA	THE DIAL	IN USE
inverted file (IVF), with product quantization	partition into cells; scan the nearest few; compress the vectors to one-byte codes	cells probed · code size	**yes**: FAISS, Milvus; the index tonight
HNSW	a layered graph of neighbours, walked greedily from an entry point, like a skip list	neighbours per node · entries explored	**yes**: most vector databases, hnswlib, pgvector
ScaNN	partition, then score with learned quantization that favours the top of the ranking	partitions probed · re-score depth	**yes**: Google; strong on the benchmarks
trees (Annoy)	random hyperplanes split the space; a forest of trees	trees · nodes visited	some; older
locality-sensitive hashing	hash functions that collide for near vectors	hash tables · bits	rarely, for vectors; the idea lives on
DiskANN	a graph laid out for an SSD, for indexes larger than memory	entries · beam width	growing, for the largest indexes

Every one trades recall for latency and memory, and every one is measured the same way: recall against exact search, at a latency. The public comparison is ann-benchmarks.com: recall against queries per second, on standard datasets, for every library.

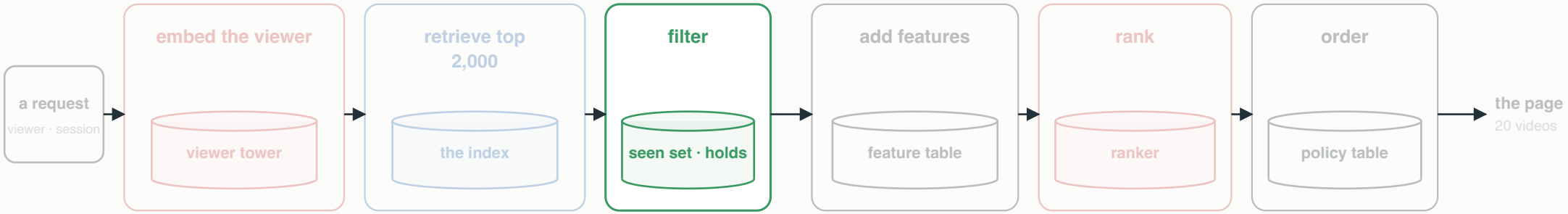
Where candidates come from



Candidate generation is several cheap generators side by side, each covering a different reason a video might belong on the page.

Each source has its own recall, measured separately. A few hundred from each, with the reason attached; merged and deduplicated, about 2,000 go to the filters.

The inference path: filter



the inference path: everything the viewer waits for, left to right, inside 300 ms

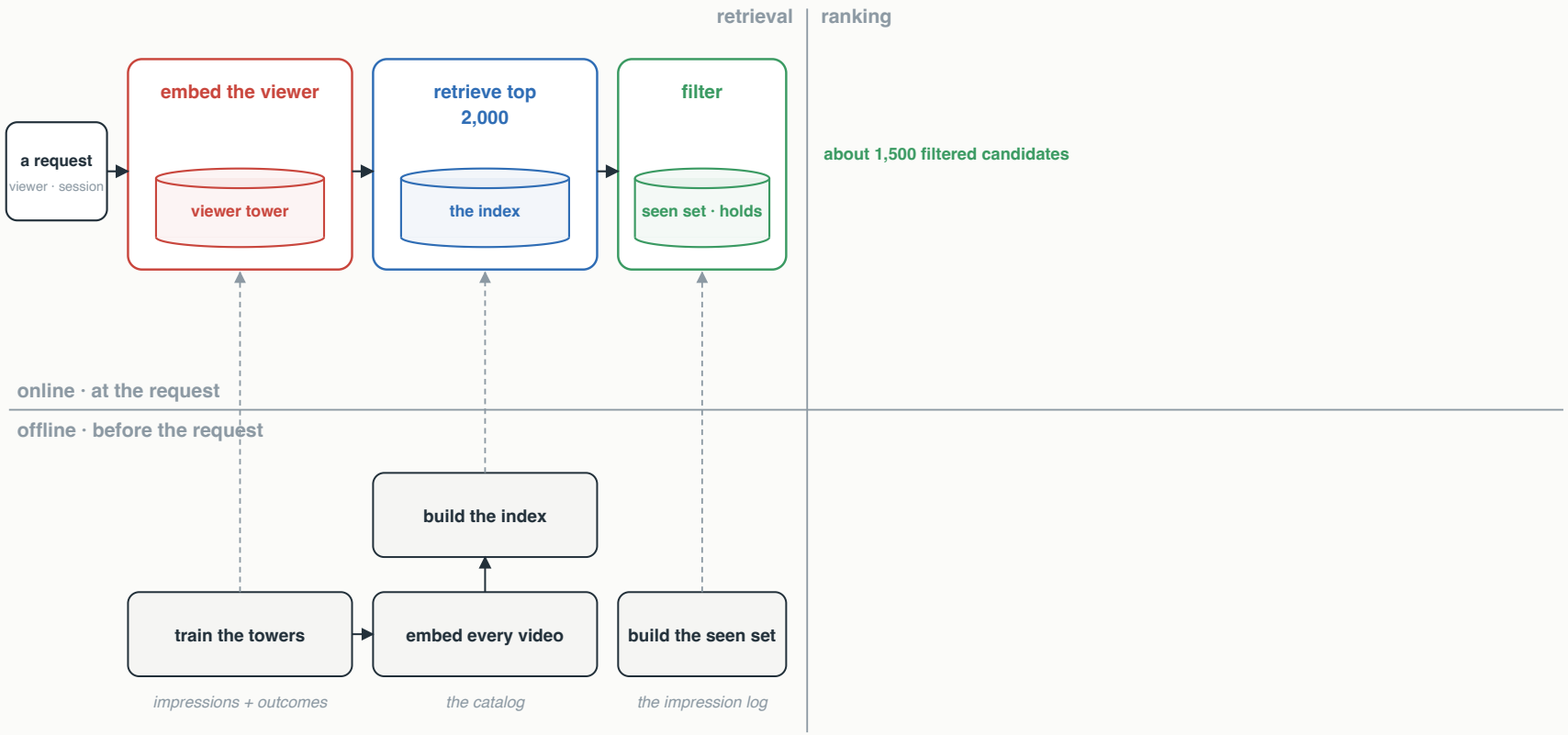
What leaves before any inference is spent on it?

Filtering, before the ranker

FILTER	HOW	WHY BEFORE THE RANKER
already shown to this viewer	a set of video ids per viewer, kept in memory and rebuilt from the impression log	an inference on a video that will be dropped is a wasted inference
under an integrity hold	a set of held video ids, pushed to every process when it changes	a held video must not appear even if the ranker loves it
not available here	region, age rating, the viewer's settings	a rule, not a preference

Code, with a test each. About a quarter of the candidates leave here; the ranker scores what remains.

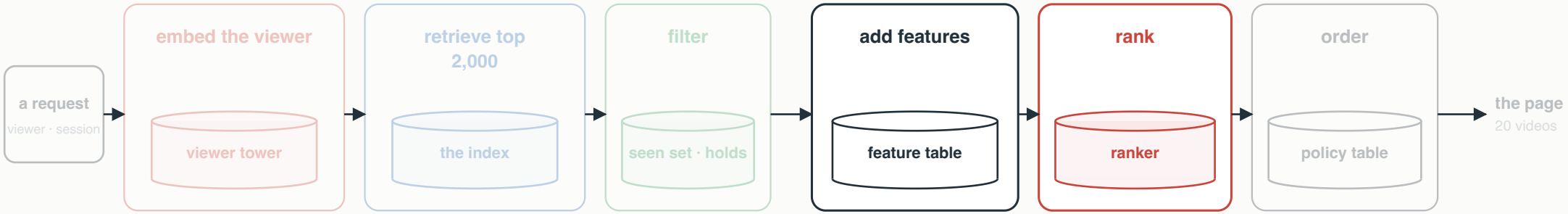
Where we are in the system



The retrieval side, built: a viewer embedded, the nearest 2,000 retrieved, the filters applied. The ranking side is still empty.

We have about 1,500 filtered candidates. Now what?

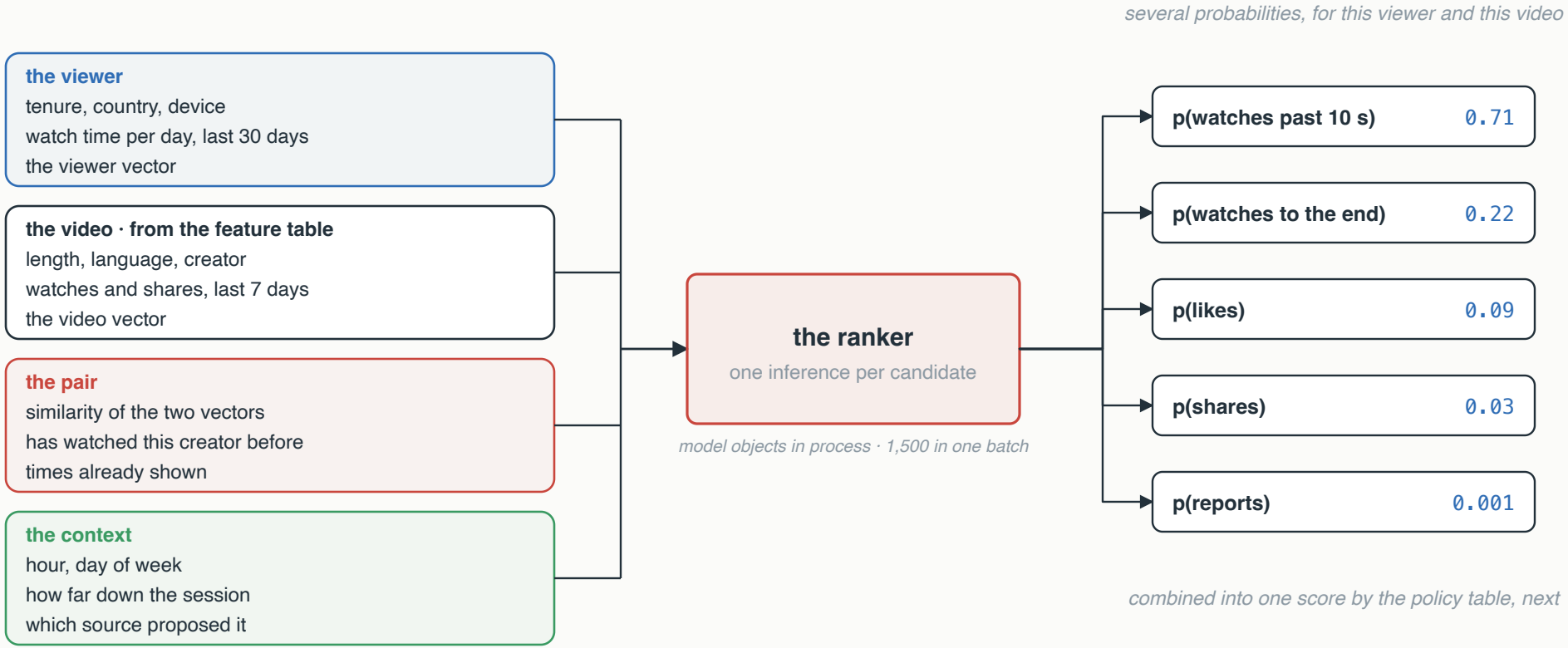
The inference path: add features, rank



the inference path: everything the viewer waits for, left to right, inside 300 ms

One inference per candidate: what does it see, and what does it produce?

The ranker



p(·): a probability for this viewer and this video, each learned from the impression log. About 50 ms for the batch of 1,500: the largest share of the budget.

The combination

Several probabilities, one order. The ranker does not decide how they combine:

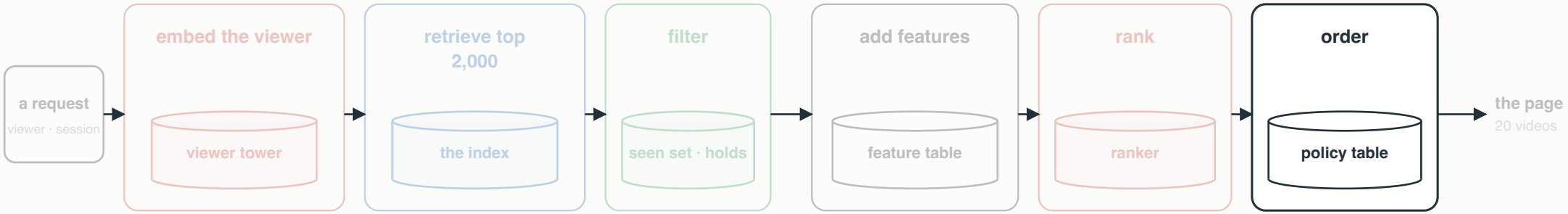
$$s = \sum_{j} w_j \, p_j$$

EVENT	*W*
watches past 10 s	1
watches to the end	3
likes	3
shares	5
reports	−50

The weights are a **requirement in a table**, owned by the product, changed without retraining, and reviewed like a policy. The ranker does not know they exist.

s: the score the page is ordered by · p_j: the ranker's probability for event *j* · w_j: the weight on that event, from the policy table.

The inference path: order



the inference path: everything the viewer waits for, left to right, inside 300 ms

200 ranked candidates become a page of 20. By what rules, and whose?

Ordering

Code, not a model. The ranker's top 200 become the page's 20 by rules anyone can read:

RULE	OWNER
no two videos from the same creator in a row	product
at most two sponsored, never in slot one	ads
one slot on a slice of pages from the **exploration** pool	the measurement team
the page is written to the impression log, with every position	the data team

Every rule is a test. The exploration slot is the data section's.

What is computed before the request, and what at it

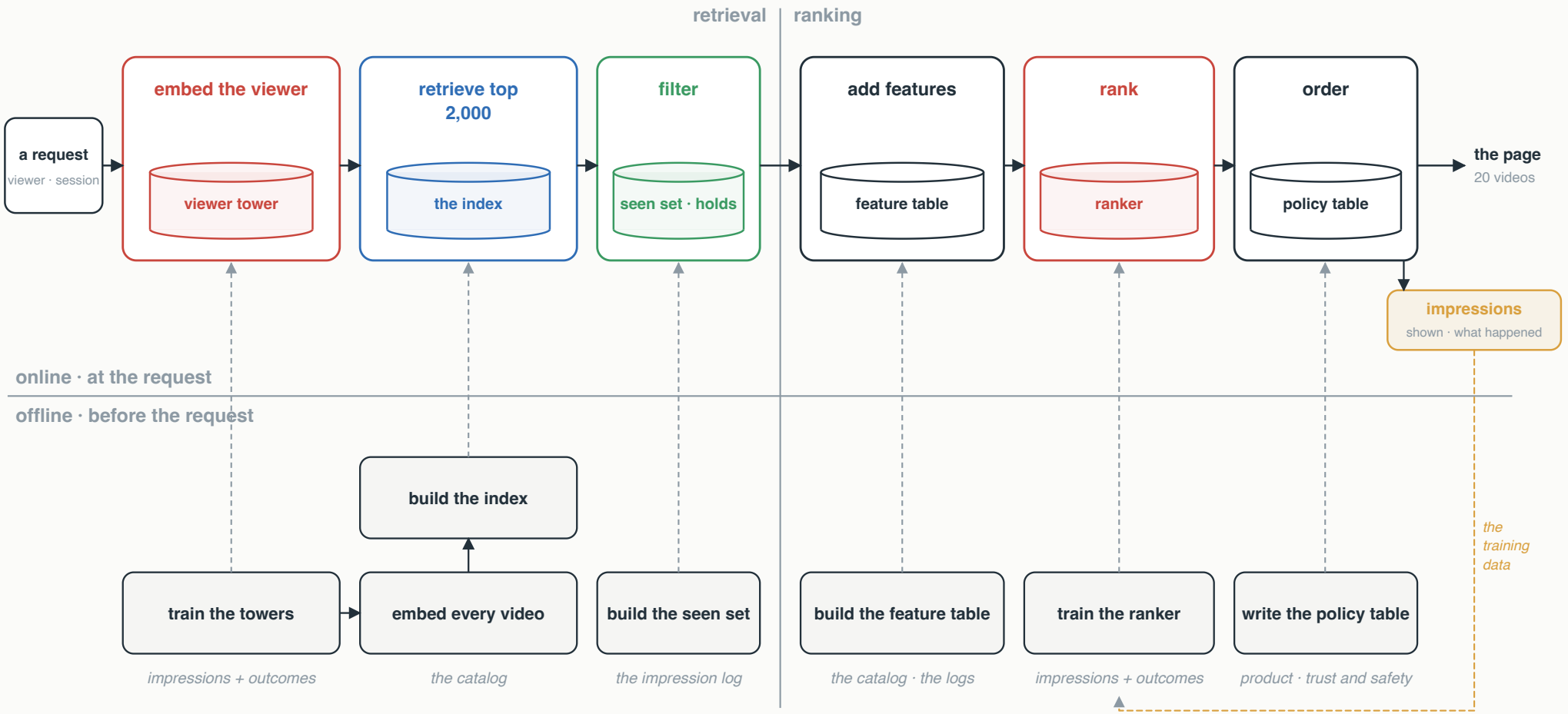
COMPUTED	BEFORE THE REQUEST	AT THE REQUEST
about every video	the vector, the index · the features · the fresh list · the holds	
about the viewer	the profile, as of last night · the seen set	the viewer vector · this session's features
about the pair		the candidates · the filters · the ranker's probabilities · the order
the seam	the index and the tables, with a version and an as-of time	read, never written
the dial	how often the index is rebuilt	cells probed · candidates ranked

As of 2016: the two networks at YouTube

THE READING (COVINGTON, ADAMS & SARGIN)	ON THE DIAGRAM
candidate generation network: the watch history and search history averaged into one input, a few layers, trained as a classifier over every video; served by nearest-neighbour lookup on its last layer	the **viewer tower**, with the averaged watch vectors as its input; the last layer's video weights are the **video vectors** in the **index**
ranking network: the impression and the watched videos embedded, language, time since last watch, number of previous impressions; trained on watch time; served on hundreds of candidates	the **ranker**: one inference per pair, with the pair features and the context; the **impression log** as its training set, with the position in it
"example age" as a feature	the **fresh** problem: without it the model prefers old videos, because old videos have more watches

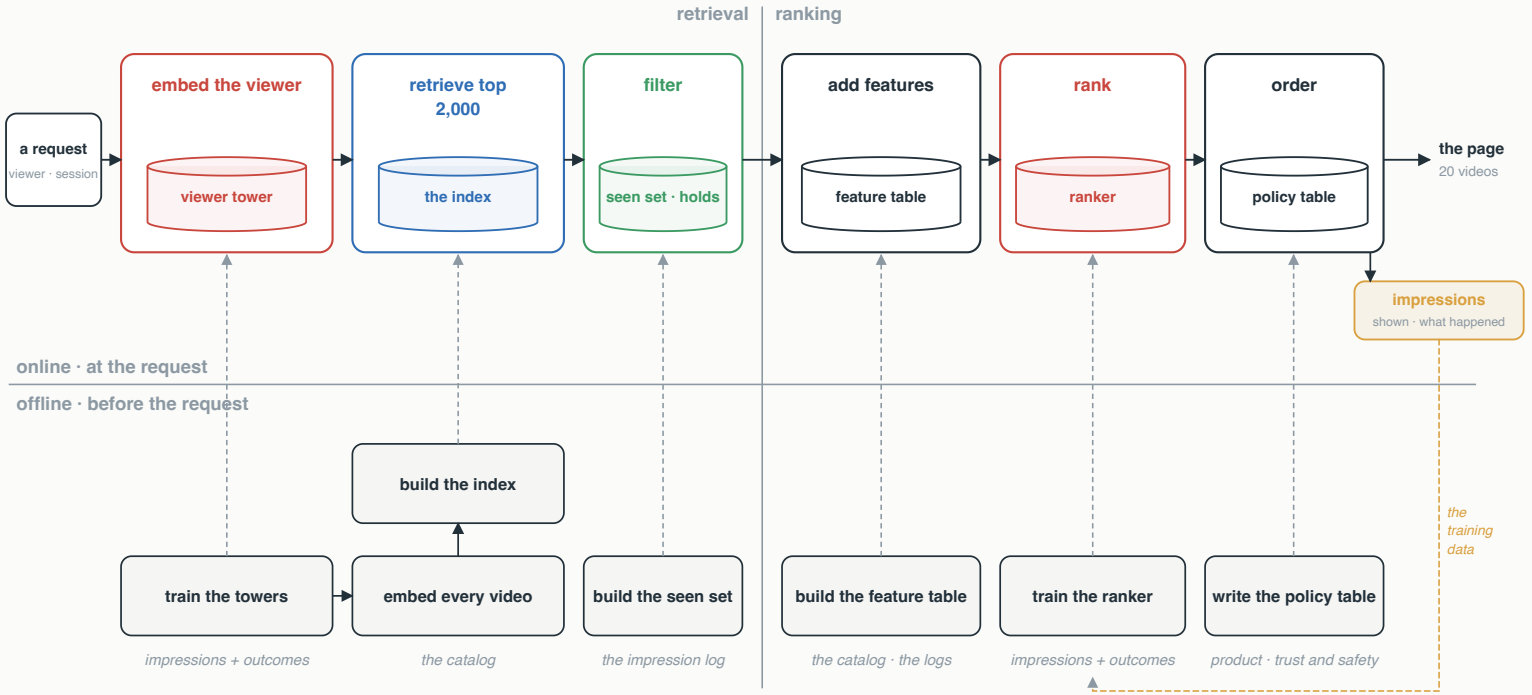
Covington, Adams & Sargin, "Deep Neural Networks for YouTube Recommendations," RecSys 2016. The paper's figures 3 and 7 are the two networks.

The ranking system, complete



The whole system. Every online stage reads an artifact that the offline side built; the impression log is what the offline side trains on.

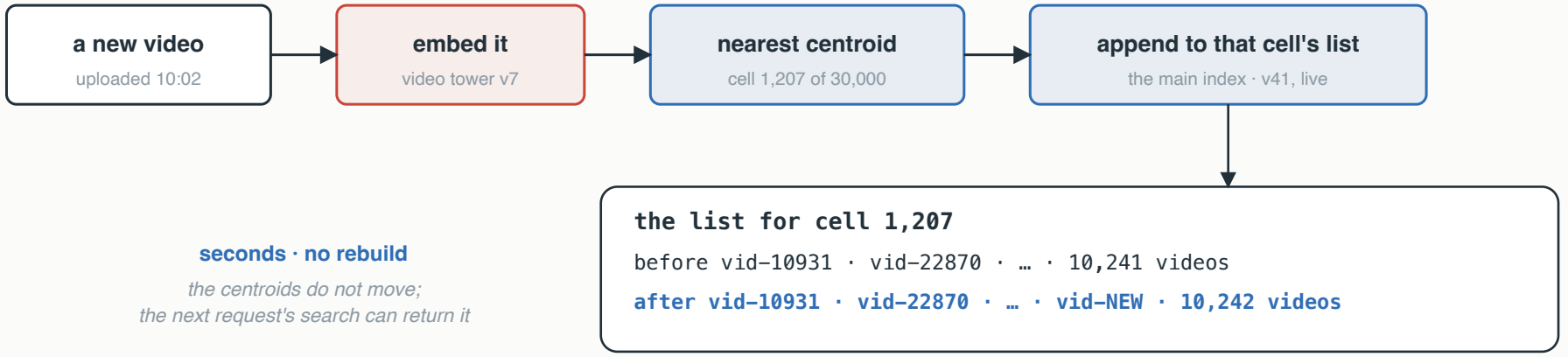
The ranking system, complete



The whole system. Every online stage reads an artifact that the offline side built; the impression log is what the offline side trains on.

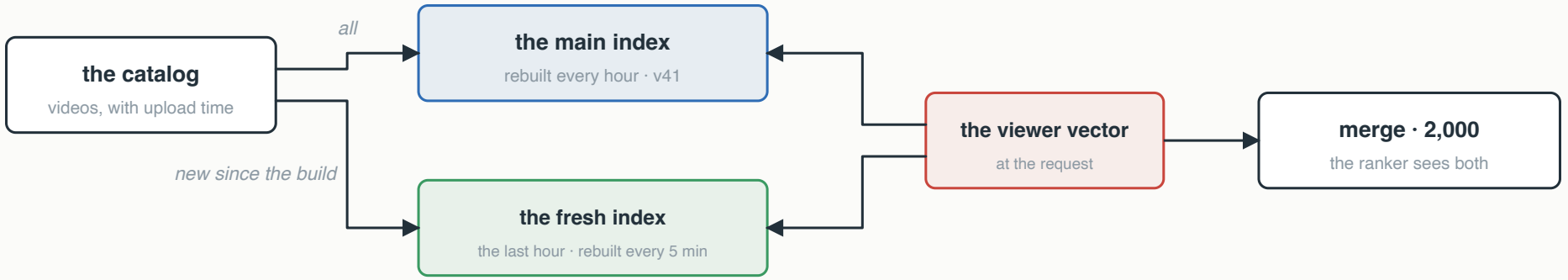
How can the system ever recommend a new video to someone, if it was never trained on that video?

Inserting a new video



A new video is an insert: its vector joins an existing cell. The cells drift from their centroids a little with every insert, which the periodic rebuild corrects.

Inserting a new video: the fresh-index alternative



A video uploaded at 10:02 is in the fresh index by 10:07 and in the main index at the next hourly build.

Both indexes carry the version of the video tower that embedded them; a vector from viewer tower v7 is only compared against v7 indexes.

The alternative, for a main index that is read-only once built: what the hourly build cannot have is kept in a small one that is cheap to rebuild often.

RANKING SYSTEMS · SECTION 2

Handling failures in a ranking system

When the ranking system fails

WHEN	WHAT RUNS	WHAT THE VIEWER SEES
the index build fails its check	the **previous index** stays in service; the pointer did not move	a feed an hour staler
the ranker exceeds its budget	the candidates in **generator order**: the index's similarity is an order too	a worse page, on time
one candidate source times out	merge **what returned** by the deadline	a page from fewer sources
the viewer has no history	the **fresh and trending** source alone	the same first page as everyone
the policy table is unreadable	**no page**, and an alarm; the app shows the last page cached	a stale page, never an unreviewed one

Everything here degrades the order, not the page.

The data

A VIDEO FEED · THE DATA

What does the system know about whether a video was a good choice?

The impression log

impressions — *one row per video shown · written by the page*

request	viewer	video	position	source	ranker	models	index
r-7f21	V-2041	vid-88	1	nn	0.71	r9 · t7	v41
r-7f21	**V-2041**	**vid-13**	**2**	**follow**	**0.66**	**r9 · t7**	**v41**
r-7f21	V-2041	vid-51	3	nn	0.64	r9 · t7	v41
…							
r-7f21	V-2041	vid-02	20	fresh	0.31	r9 · t7	v41

join

outcomes — *one row per event*

request	video	event	seconds
r-7f21	vid-88	skip	2
r-7f21	**vid-13**	**watch**	**41**
r-7f21	**vid-13**	**like**	
r-7f21	vid-51	skip	1

one training row: viewer V-2041 · video vid-13 · shown at position 2 by the follow source · scored 0.71 by ranker r9 · watched 41 s, liked

the label is what happened to a video that was shown, in the slot it was shown in, by the ranker that chose it

The impression log is the training set. Every row says what was shown, where, by which models, and what happened next.

What trains the towers

THE TOWERS	
the rows	pairs from the impression log: a viewer's recent watches, and the video they watched next
the signal	the two towers are fitted so that the viewer's vector lands near the vector of the video they went on to watch, and far from videos sampled at random
what "near" comes to mean	watched by the same people, after the same histories. Not "about the same thing"
what is missing	a video nobody has watched has no row, and lands nowhere useful until it does

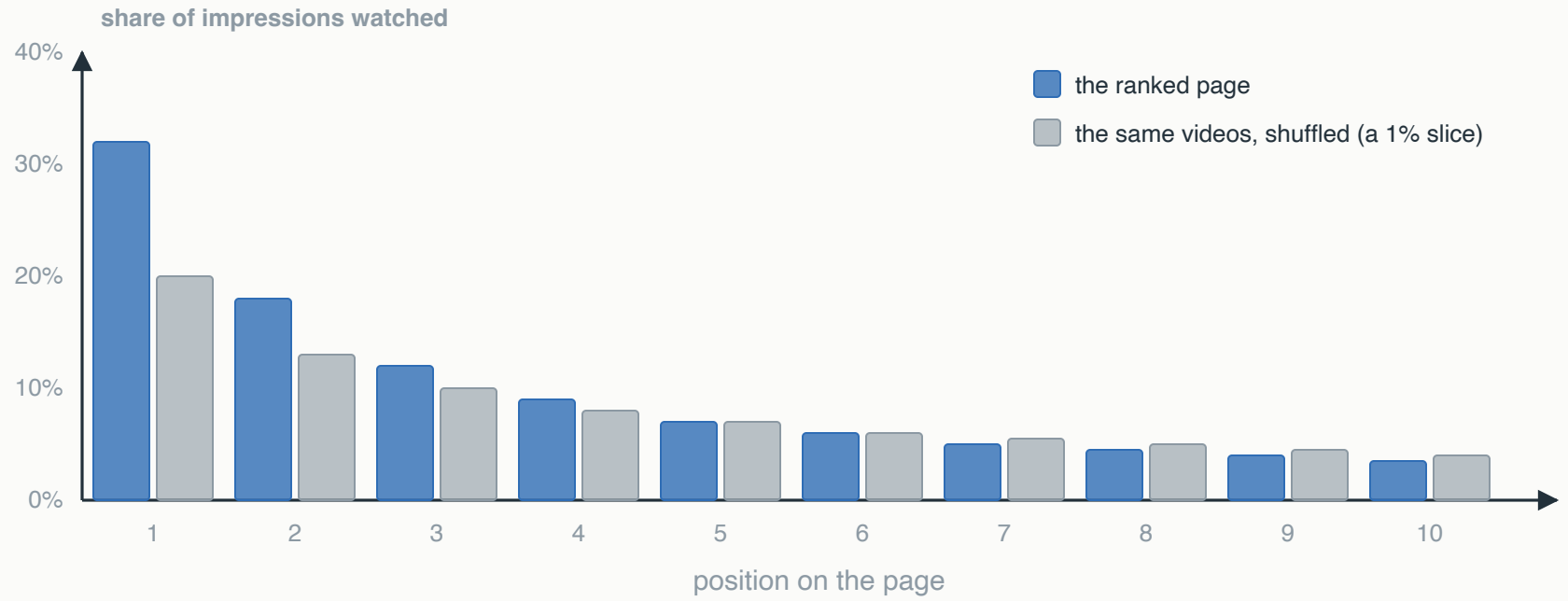
The same log trains the retriever, the ranker, and the next version of both.

What a watch is, and what it is not

A WATCH SAYS	A WATCH DOES NOT SAY
this viewer, shown this video, **in this slot**, chose it	that it was the best video for them
among the twenty shown	anything about the 299,999,980 not shown
as chosen by **ranker r9**, with the weights of that day	what a different ranker's page would have earned

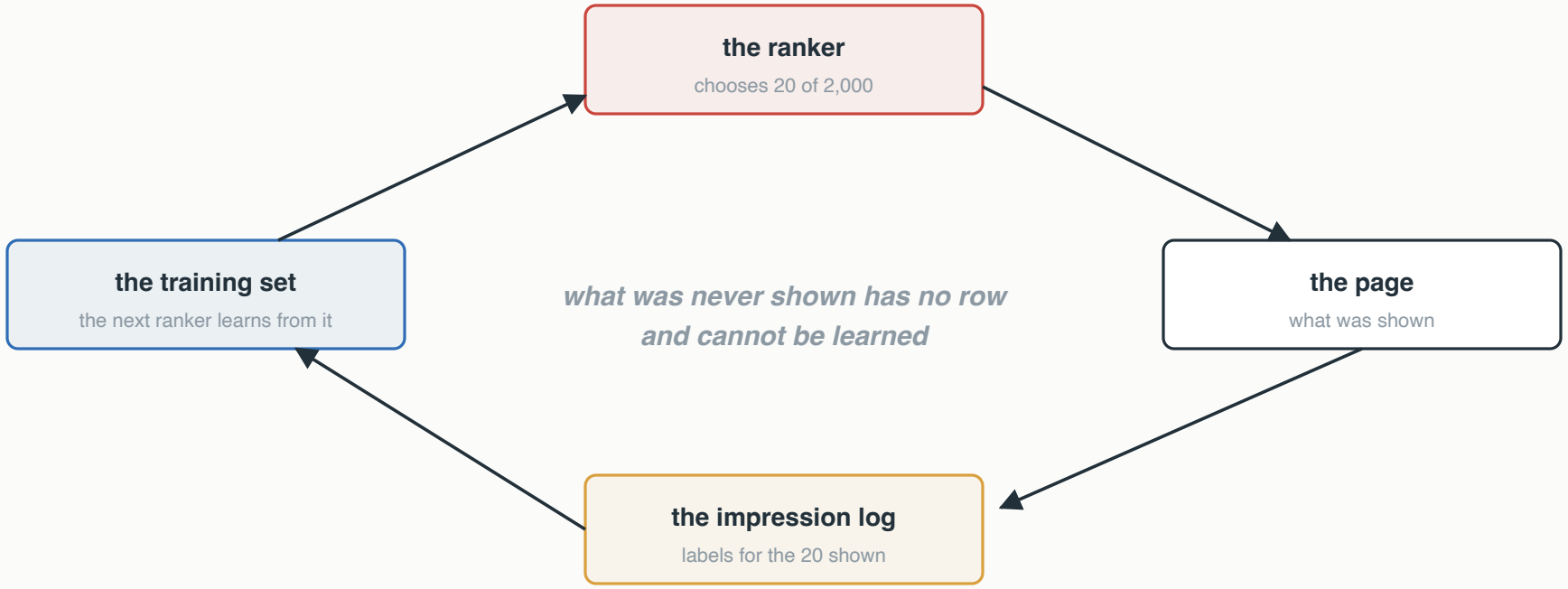
The label is implicit: an event that happened to what the system chose to show, where it chose to show it.

Position bias



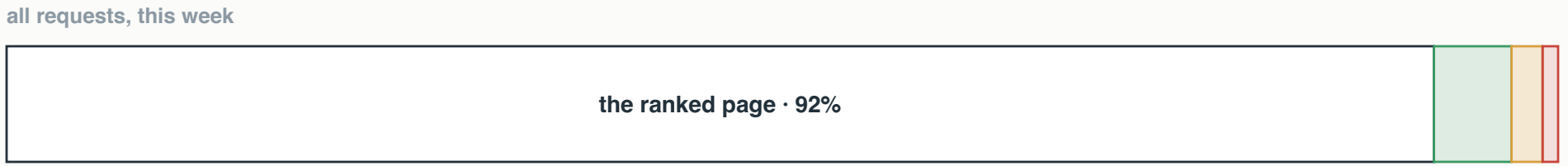
Slot one is watched more even when it holds a random video. A watch in slot 1 is a weaker label than one in slot 9. Illustrative.

The feedback loop



The loop. Outputs become inputs: the next training set is the last ranker's choices.

Resolution: slices set aside on purpose



exploration slot · 5% of pages
one slot filled from candidates the ranker scored low, or at random: a label for what it would not have shown
slice = explore, on that row

holdout · 2% of viewers
the previous ranker, for the whole quarter: the long-run effect of the current one
slice = holdout, on every row

shuffled · 1% of requests
the twenty in random order: the slot's own effect, so position can be corrected
slice = shuffled, on every row

Each slice is flagged on its impression rows at serving time; every number in the measurement section says which slice it came from.

The training set

THE TRAINING SET	
the query	impressions joined with outcomes, on request and video, as of a date
one row	viewer, video, position, source, the ranker's inputs as served, the versions, and what happened
the label	the events, with their weights from the policy table of that day
kept out	the outcomes of videos the viewer watched *later*, from a different page: known after the impression
frozen	to files with a hash and a number; the next ranker carries the number

Log what was served. Train on what was served.

The index is a dataset

THE INDEX CARRIES	WHY
a version, and the run that built it	the request path loads by version; a failed build leaves the old one current
the tower version that embedded it	a viewer vector from another version is a wrong answer that looks right
the catalog as-of time	freshness: the newest video it can return
its count, and its recall against exact search	a build that lost a tenth of the catalog, or a tenth of its recall, is refused

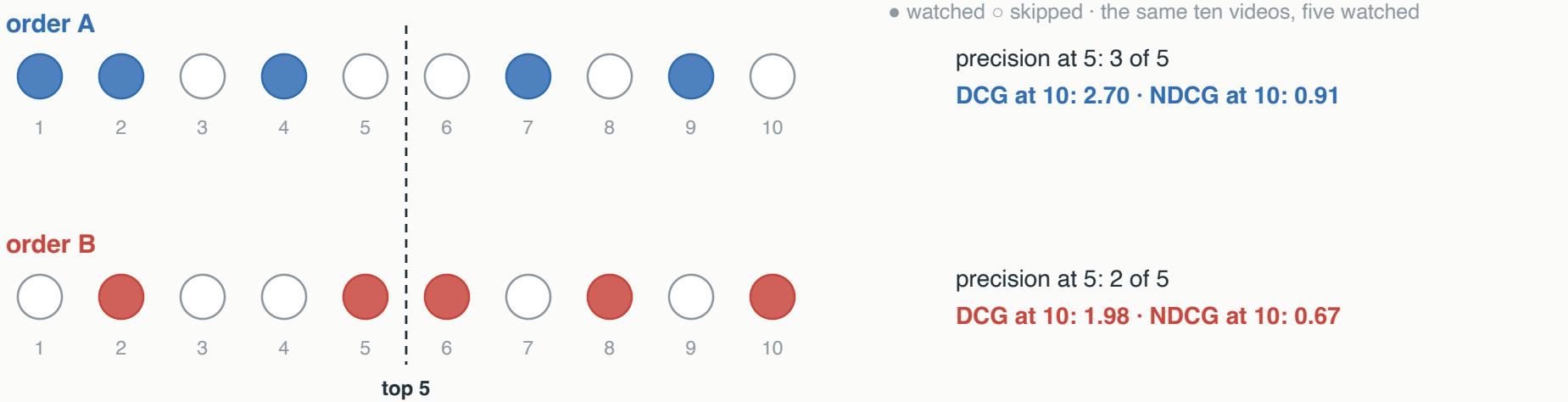
FOR CODE	FOR THE INDEX
version control	a version, a tower version, an as-of
a unit test	the count and the recall check, after every build
the build pins a commit	the pointer pins the **index and the towers together**

The measurement

What we measure, and why

WHAT	THE MEASURES	WHY
ranking performance	recall at *k* per stage · DCG and NDCG on logged pages	the order is the product; each stage caps the next
ranking latency	p99 per stage, and end to end	the page has 300 ms, and the ranker spends most of it
A/B testing	watch time per session, two rankers, random assignment, the difference against the A/A floor	offline, the old ranker grades the new one; only real viewers on the new pages can decide
deployment guardrails	reports, crashes, latency, revenue, the loop: each with its own floor	a primary that rises while something else falls must not ship

The problem: two orders of the same page



Precision at 10 is 5 of 10 for both. Only a measure that discounts by position tells them apart.

ideal DCG at 10: the five watched in the first five slots, 2.95

Two orders of the same page. The viewer watched the same five videos; one order put them where they would be seen.

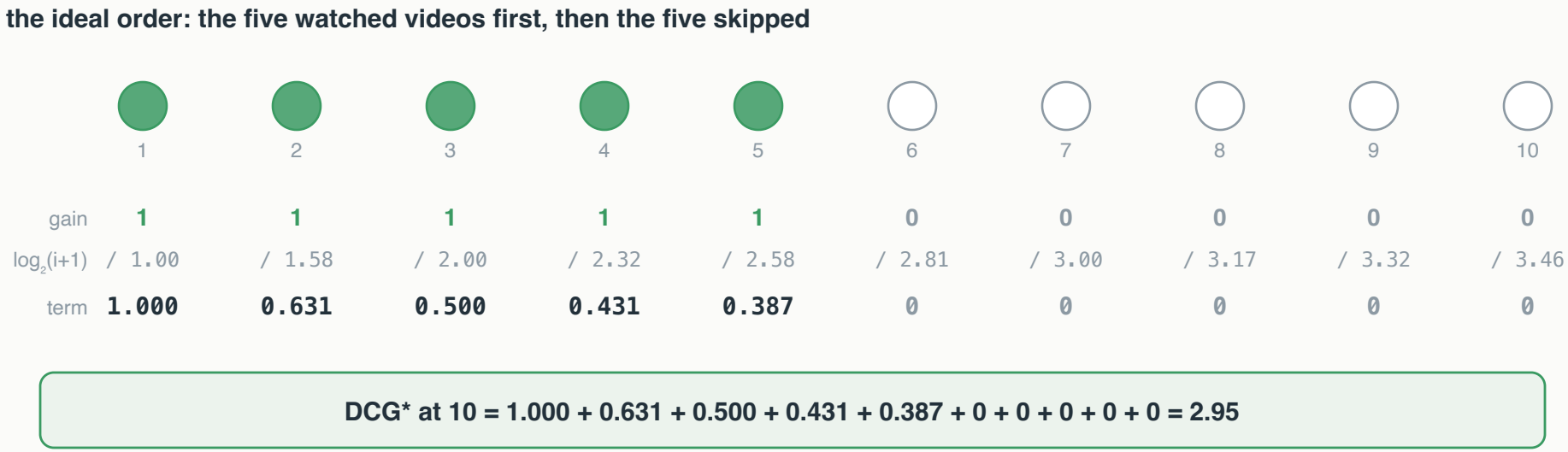
Resolution: discount by position

$$\mathrm{DCG@}k = \sum_{i=1}^{k} \frac{g_i}{\log_2(i+1)} \qquad \mathrm{NDCG@}k = \frac{\mathrm{DCG@}k}{\mathrm{DCG}^*\mathrm{@}k}$$

THE SAME PAGE	ORDER A	ORDER B
watched at positions	1, 2, 4, 7, 9	2, 5, 6, 8, 10
DCG at 10	1 + 0.63 + 0.43 + 0.33 + 0.30 = **2.70**	0.63 + 0.39 + 0.36 + 0.32 + 0.29 = **1.98**
NDCG at 10	2.70 / 2.95 = **0.91**	1.98 / 2.95 = **0.67**

- g_i: the gain of the video at position i: 1 if watched, 0 if not; or the watch seconds
- $\log_2(i+1)$: the discount: 1 at the top, falling slowly down the page
- DCG*: the DCG of the best order of the same videos, 2.95 here; so NDCG is 1 for a perfect page
- k: the page length

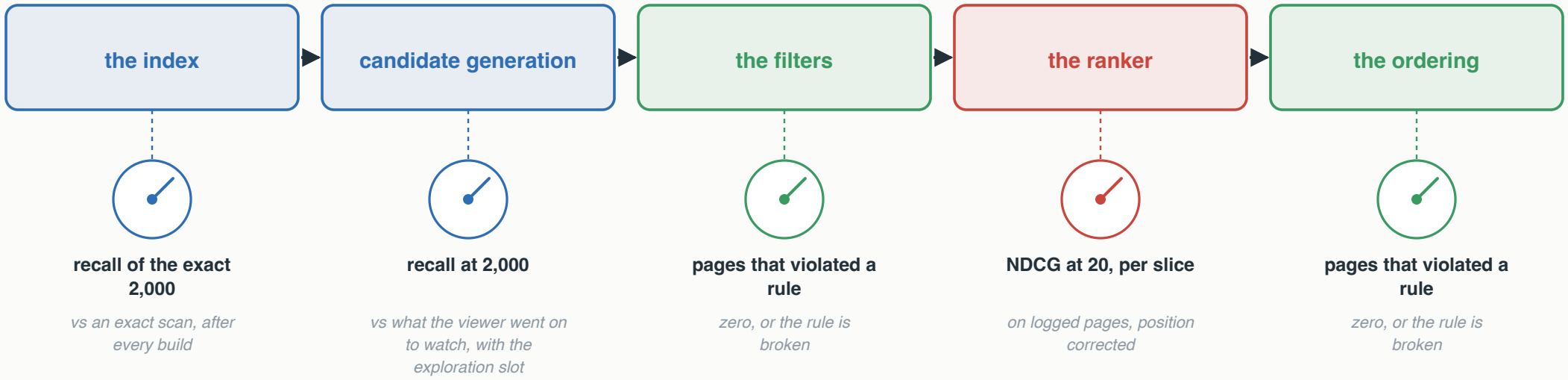
Resolution, continued: the ideal order, DCG*



The ceiling for a page with five watched videos out of ten. A page with a different number of watched videos has a different ceiling.

The same ten videos, reordered so that the five the viewer watched come first. DCG* is the DCG of that order. Order A scored 2.70 of a possible 2.95; order B, 1.98.

Measuring the stages separately



The shape says where to put the meter: one per stage, each against its own reference, none of them the page's watch time.

Each stage's ceiling is the recall of the stage before it. A ranker cannot place a video the index did not return.

A VIDEO FEED · THE MEASUREMENT

A new ranker is fitted. How do we know whether it is better?

The problem: what offline measurement cannot see

A new ranker, r10, is fitted. Its NDCG on last week's impressions is **0.74 against r9's 0.71**. Last week's impressions are r9's choices.

ranker r9 · the page that was shown

vid-88	skipped
vid-13	watched
vid-51	skipped
vid-07	watched
vid-32	skipped
vid-19	skipped

ranker r10 · the page it would have shown

vid-13	watched
vid-62	no label
vid-07	watched
vid-91	no label
vid-51	skipped
vid-44	no label

scoring r10 offline

vid-13, vid-07: watched, counted
vid-51: skipped, counted
vid-62, vid-91, vid-44: never shown, so no label, so counted as unwatched

r10 is penalized exactly where it differs from r9

the shuffled and exploration rows narrow this; they do not close it

Offline, the old ranker grades the new one, on the old ranker's pages. Only the videos r9 chose to show have labels.

Resolution: the A/B test

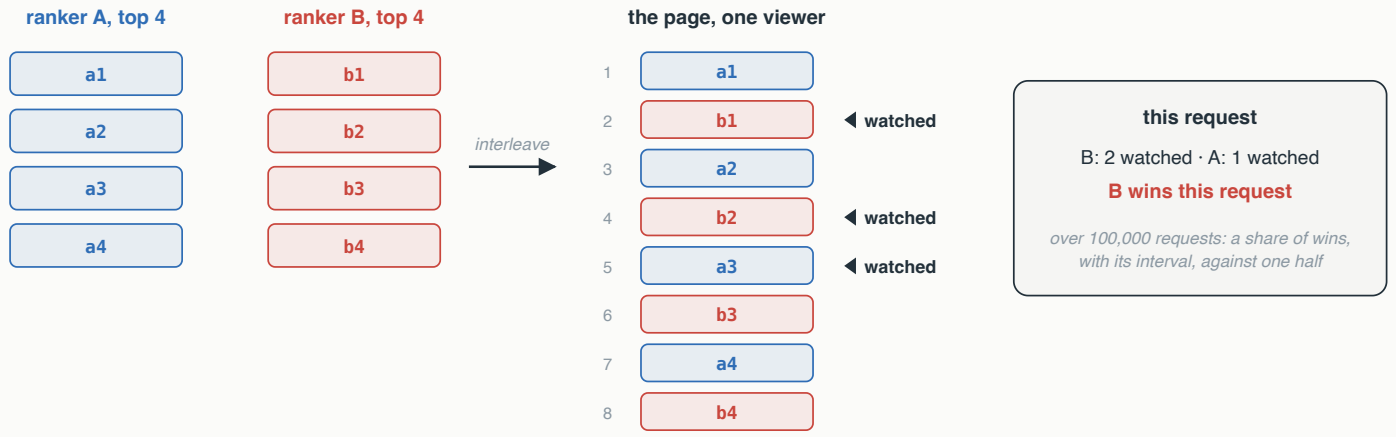




Random assignment makes the two arms alike in everything but the ranker. The difference, with its interval, is the measurement.

Resolution, continued: interleaving

An A/B test needs a million viewers an arm for a week, because **viewers differ far more than rankers do**. Interleaving shows both rankers to the **same viewer on the same page**, so the viewer cancels out.



Interleaving: both rankers on every page, alternating positions, so each request is a paired comparison and position bias falls on both alike.

A paired comparison per request: a decision in a day on a small slice of traffic, not a week on half of it. It answers which order viewers prefer; the A/B test still decides the release.

Guardrails, and the release gate

THE GATE	THE MEASURE	MOVES THE RELEASE WHEN
primary	watch time per session, per viewer	the interval clears the A/A floor
guardrails	reports per 1,000 impressions · p99 latency of the page · crashes · time to first video · revenue per session	any one moves against, by more than its own floor
slices	new viewers · viewers under 18 · each country	the primary helped, hurt or cannot tell, per slice
the loop	the share of impressions from creators uploading this week	not narrowing

A release is a primary that cleared the floor and a guardrail table with nothing in it.

The pipeline's own measures

MEASURE	WHAT IT ASKS	REQUIREMENT
index freshness	the age of the newest video the current index can return	under 10 minutes, with the fresh index
index recall	against exact search, after every build	over 85% at the cells probed
version match	the tower version of the index equals the tower version serving viewer vectors	always; a test, not a rate
page latency	p99, end to end, and per stage	under 300 ms
completeness	the share of pages with twenty videos	over 99.9%
impression coverage	the share of pages rendered that have impression rows	100%; the training set is this table

None of these needs a label; they are measured as the system runs.

The measurement so far

SYSTEM	WHAT IS MEASURED	THE INSTRUMENT
any model call	a rate on a slice, with its interval; the noise floor	the golden set; repeated runs
a change to a system	helped · hurt · cannot tell, per slice	paired differences
a judge, human or model	agreement with a reference	the four counts; κ
a scoring system	precision and recall at *k*, AUC, calibration, on a holdout; stored = recomputed; freshness, completeness	the inference table joined with the labels, 90 days later
a ranking system	**recall at *k* per stage; NDCG on logged pages; the A/B test against the A/A floor; interleaving; guardrails; the index against exact search**	**the impression log, with its slices; random assignment of viewers**

FAILURE OF THE WEEK, REVISITED

Which question did they never ask?

- **How is it measured?** The primary metric was the label. No guardrail in the gate measured what Facebook said it wanted; no survey slice sat beside the engagement rate.
- **How is the data handled?** The weights were a policy table, like tonight's combination. Changing them changed the label of every row written after; the training set after the change was produced by people who had learned the weights.
- **What is the architecture** was the only question answered, and it was answered well.

What a fix would have looked like:

- a slow label beside the fast one: a survey answer, joined to the impressions it was about
- the guardrail table, with the survey rate in it, able to stop a release
- the loop measured: what producers changed after the weights changed

DESIGN STUDIO

Design the simplified Amazon store front

A customer opens the store's home page and sees fifty products chosen for them. Design the ranking system behind the grid: the architecture, the data, the measurement. Only what is in stock; not what they already bought; a vector database; a way to train the models and a way to get the training sets.

The full prompt is on the course site.

LAB · GITHUB.COM/AISE-STTHOMAS/PUBLISH-THE-API

Publish the API

A function holding the Gemini key with two routes, a chat turn and the triage step, an API gateway in front of it, and a DynamoDB table behind it: a public URL with a Swagger page you can open on your phone, a token for the model calls, every call as a row in the table and a line in the log, and a classmate's API to call. Leave it running: the next project starts from it.

The instructions are in the repository's LAB.md.

Before you go

- **Reading for next week:** Lewis et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks* (2020), §1–3 · Zheng et al., *Judging LLM-as-a-Judge* (2023), §1–3
- **Optional, on tonight's system:** Covington, Adams & Sargin, *Deep Neural Networks for YouTube Recommendations* (RecSys 2016): the two networks, and the actual features each one was given
- **Project 1** is due; **Project 2** will be assigned shortly.
- Next week: the retrieval-grounded assistant, and the lab builds the index it reads.